



LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping

Tixiao Shan, Brendan Englot, Drew Meyers, Wei Wang, Carlo Ratti, and Daniela Rus

Abstract— We propose a framework for tightly-coupled lidar inertial odometry via smoothing and mapping, LIO-SAM, that achieves highly accurate, real-time mobile robot trajectory estimation and map-building. LIO-SAM formulates lidar-inertial odometry atop a factor graph, allowing a multitude of relative and absolute measurements, including loop closures, to be incorporated from different sources as factors into the system. The estimated motion from inertial measurement unit (IMU) pre-integration de-skews point clouds and produces an initial guess for lidar odometry optimization. The obtained lidar odometry solution is used to estimate the bias of the IMU. To ensure high performance in real-time, we marginalize old lidar scans for pose optimization, rather than matching lidar scans to a global map. Scan-matching at a local scale instead of a global scale significantly improves the real-time performance of the system, as does the selective introduction of keyframes, and an efficient sliding window approach that registers a new keyframe to a fixed-size set of prior “sub-keyframes.” The proposed method is extensively evaluated on datasets gathered from three platforms over various scales and environments.

I. INTRODUCTION

State estimation, localization and mapping are fundamental prerequisites for a successful intelligent mobile robot, required for feedback control, obstacle avoidance, and planning, among many other capabilities. Using vision-based and lidar-based sensing, great efforts have been devoted to achieving high-performance real-time simultaneous localization and mapping (SLAM) that can support a mobile robot’s six degree-of-freedom state estimation. Vision-based methods typically use a monocular or stereo camera and triangulate features across successive images to determine the camera motion. Although vision-based methods are especially suitable for place recognition, their sensitivity to initialization, illumination, and range make them unreliable when they alone are used to support an autonomous navigation system. On the other hand, lidar-based methods are largely invariant to illumination change. Especially with the recent availability of long-range, high-resolution 3D lidar, such as the Velodyne VLS-128 and Ouster OS1-128, lidar becomes more suitable to directly capture the fine details of an environment in 3D space. Therefore, this paper focuses on lidar-based state estimation and mapping methods.

Many lidar-based state estimation and mapping methods have been proposed in the last two decades. Among them, the

lidar odometry and mapping (LOAM) method proposed in [1] for low-drift and real-time state estimation and mapping is among the most widely used. LOAM, which uses a lidar and an inertial measurement unit (IMU), achieves state-of-the-art performance and has been ranked as the top lidar-based method since its release on the KITTI odometry benchmark site [2]. Despite its success, LOAM presents some limitations - by saving its data in a global voxel map, it is often difficult to perform loop closure detection and incorporate other absolute measurements, e.g., GPS, for pose correction. Its online optimization process becomes less efficient when this voxel map becomes dense in a feature-rich environment. LOAM also suffers from drift in large-scale tests, as it is a scan-matching based method at its core.

In this paper, we propose a framework for tightly-coupled lidar inertial odometry via smoothing and mapping, LIO-SAM, to address the aforementioned problems. We assume a nonlinear motion model for point cloud de-skew, estimating the sensor motion during a lidar scan using raw IMU measurements. In addition to de-skewing point clouds, the estimated motion serves as an initial guess for lidar odometry optimization. The obtained lidar odometry solution is then used to estimate the bias of the IMU in the factor graph. By introducing a global factor graph for robot trajectory estimation, we can efficiently perform sensor fusion using lidar and IMU measurements, incorporate place recognition among robot poses, and introduce absolute measurements, such as GPS positioning and compass heading, when they are available. This collection of factors from various sources is used for joint optimization of the graph. Additionally, we marginalize old lidar scans for pose optimization, rather than matching scans to a global map like LOAM. Scan-matching at a local scale instead of a global scale significantly improves the real-time performance of the system, as does the selective introduction of keyframes, and an efficient sliding window approach that registers a new keyframe to a fixed-size set of prior “sub-keyframes.” The main contributions of our work can be summarized as follows:

- A tightly-coupled lidar inertial odometry framework built atop a factor graph, that is suitable for multi-sensor fusion and global optimization.
- An efficient, local sliding window-based scan-matching approach that enables real-time performance by registering selectively chosen new keyframes to a fixed-size set of prior sub-keyframes.
- The proposed framework is extensively validated with tests across various scales, vehicles, and environments.

T. Shan, D. Meyers, W. Wang, and C. Ratti are with the Department of Urban Studies and Planning, Massachusetts Institute of Technology, USA, {shant, drewm, wweiwang, ratti}@mit.edu.

B. Englot is with the Department of Mechanical Engineering, Stevens Institute of Technology, USA, benglot@stevens.edu.

T. Shan, W. Wang, and D. Rus are with the Computer Science & Artificial Intelligence Laboratory, Massachusetts Institute of Technology, USA, {shant, wweiwang, rus}@mit.edu.



LIO-SAM：透過平滑與建圖實現緊密耦合的雷射慣性里程計

Tixiao Shan、Brendan Englot、Drew Meyers、Wei Wang、Carlo Ratti 和 Daniela Rus

摘要— 我們提出一個透過平滑與建圖實現緊密耦合雷射雷達慣性里程計的框架，LIO-SAM，它能實現高精確度、即時的行動機器人軌跡估計和地圖建構。LIO-SAM 將雷射雷達慣性里程計建立在因子圖之上，允許將多種相對和絕對測量（包括迴圈閉合）作為因子從不同來源整合到系統中。慣性測量單元 (IMU) 預積分的估計運動能消除點雲的傾斜，並為雷射雷達里程計最佳化提供初始猜測。所獲得的雷射雷達里程計解用於估計 IMU 的偏差。為了確保即時高效能，我們對舊的雷射雷達掃描進行邊緣化以進行姿態最佳化，而不是將雷射雷達掃描與全域地圖進行匹配。在局部而非全域範圍內進行掃描匹配顯著提高了系統的即時效能，選擇性地引入關鍵影格以及高效的滑動視窗方法（將新關鍵影格註冊到固定大小的先前「子關鍵影格」集合）也同樣如此。所提出的方法在從三個平台在不同規模和環境中收集的資料集上進行了廣泛評估。

I. 簡介

狀態估計、定位和建圖是成功智慧行動機器人的基本先決條件，是回饋控制、避障和規劃等許多其他能力的必要條件。透過基於視覺和雷射雷達的感測，人們投入了巨大的努力來實現高效能的即時同步定位與建圖 (SLAM)，以支援行動機器人的六自由度狀態估計。基於視覺的方法通常使用單眼或立體攝影機，並透過連續影像之間的特徵三角測量來確定攝影機運動。儘管基於視覺的方法特別適合於地點識別，但它們對初始化、照明和範圍的敏感性使得它們在單獨用於支援自主導航系統時不可靠。另一方面，基於雷射雷達的方法在很大程度上不受照明變化的影響。特別是隨著最近長距離、高解析度 3D 雷射雷達（例如 Velodyne VLS-128 和 Ouster OS1-128）的出現，雷射雷達變得更適合直接捕捉 3D 空間中環境的精細細節。因此，本文重點介紹基於雷射雷達的狀態估計和建圖方法。

在過去的二十年中，已經提出了許多基於光達的狀態估計和地圖建構方法。其中，

[1] 中提出的雷射雷達里程計和映射 (LOAM) 方法，用於低漂移和即時狀態估計和映射，是應用最廣泛的方法之一。LOAM 使用雷射雷達和慣性測量單元 (IMU)，實現了最先進的性能，自其在 KITTI 里程計基準測試網站 [2] 上釋出以來，一直被評為頂級的基於雷射雷達的方法。儘管 LOAM 取得了成功，但它也存在一些限制——透過將其資料儲存在全域體素地圖中，通常難以執行迴路閉合偵測並結合其他絕對測量，例如 GPS，進行姿態校正。當這個體素地圖在特徵豐富的環境中變得密集時，其線上最佳化過程效率會降低。LOAM 在大規模測試中也存在漂移問題，因為其核心是一種基於掃描匹配的方法。

在本文中，我們提出了一個透過平滑和映射實現緊密耦合雷射雷達慣性里程計的框架，即 LIO-SAM，以解決上述問題。我們假設點雲去偏斜的非線性運動模型，使用原始 IMU 測量估計雷射雷達掃描期間的感測器運動。除了去偏斜點雲之外，估計的運動還作為雷射雷達里程計最佳化的初始猜測。然後，獲得的雷射雷達里程計解決方案用於估計因子圖中 IMU 的偏差。透過引入用於機器人軌跡估計的全域因子圖，我們可以有效地使用雷射雷達和 IMU 測量進行感測器融合，在機器人姿態之間引入位置識別，並在可用時引入絕對測量，例如 GPS 定位和羅盤航向。來自各種來源的這些因子集合用於圖的聯合最佳化。此外，我們將舊的雷射雷達掃描邊緣化以進行姿態最佳化，而不是像 LOAM 那樣將掃描與全域地圖匹配。在局部而非全域範圍內進行掃描匹配顯著提高了系統的即時性能，選擇性地引入關鍵影格以及將新關鍵影格註冊到固定大小的先前「子關鍵影格」集合的有效滑動視窗方法也如此。我們工作的主要貢獻可歸納如下：

- 一個建立在因子圖之上的緊密耦合雷射雷達慣性里程計框架，適用於多感測器融合和全域最佳化。
- 一種高效的、基於局部滑動視窗的掃描匹配方法，透過選擇性選擇的新關鍵影格註冊到固定大小的先前子關鍵影格集合來實現即時性能。
- 所提出的框架已透過各種規模、車輛和環境的測試進行了廣泛驗證。

T. Shan、D. Meyers、W. Wang 和 C. Ratti 任職於美國麻省理工學院城市研究與規劃學系，電子郵件：{shant, drewm, wweiwang, ratti}@mit.edu。

B. Englot 任職於美國史蒂文斯理工學院機械工程學系，電子郵件：benglot@stevens.edu。

T. Shan、W. Wang 和 D. Rus 任職於美國麻省理工學院電腦科學與人工智慧實驗室，電子郵件：{shant, wweiwang, rus}@mit.edu。

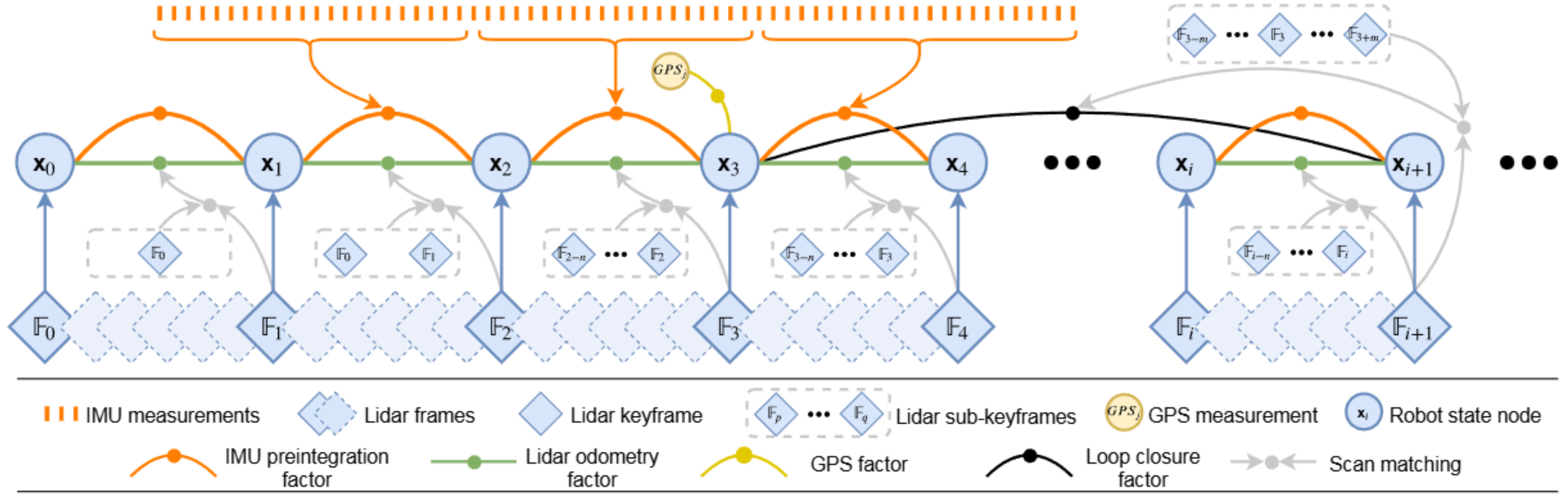


Fig. 1: The system structure of LIO-SAM. The system receives input from a 3D lidar, an IMU and optionally a GPS. Four types of factors are introduced to construct the factor graph.: (a) IMU preintegration factor, (b) lidar odometry factor, (c) GPS factor, and (d) loop closure factor. The generation of these factors is discussed in Section III.

II. RELATED WORK

Lidar odometry is typically performed by finding the relative transformation between two consecutive frames using scan-matching methods such as ICP [3] and GICP [4]. Instead of matching a full point cloud, feature-based matching methods have become a popular alternative due to their computational efficiency. For example, in [5], a plane-based registration approach is proposed for real-time lidar odometry. Assuming operations in a structured environment, it extracts planes from the point clouds and matches them by solving a least-squares problem. A collar line-based method is proposed in [6] for odometry estimation. In this method, line segments are randomly generated from the original point cloud and used later for registration. However, a scan's point cloud is often skewed because of the rotation mechanism of modern 3D lidar, and sensor motion. Solely using lidar for pose estimation is not ideal since registration using skewed point clouds or features will eventually cause large drift.

Therefore, lidar is typically used in conjunction with other sensors, such as IMU and GPS, for state estimation and mapping. Such a design scheme, utilizing sensor fusion, can typically be grouped into two categories: loosely-coupled fusion and tightly-coupled fusion. In LOAM [1], IMU is introduced to de-skew the lidar scan and give a motion prior for scan-matching. However, the IMU is not involved in the optimization process of the algorithm. Thus LOAM can be classified as a loosely-coupled method. A lightweight and ground-optimized lidar odometry and mapping (LeGO-LOAM) method is proposed in [7] for ground vehicle mapping tasks [8]. Its fusion of IMU measurements is the same as LOAM. A more popular approach for loosely-coupled fusion is using extended Kalman filters (EKF). For example, [9]-[13] integrate measurements from lidar, IMU, and optionally GPS using an EKF in the optimization stage for robot state estimation.

Tightly-coupled systems usually offer improved accuracy and are presently a major focus of ongoing research [14]. In [15], preintegrated IMU measurements are exploited for de-skewing point clouds. A robocentric lidar-inertial state estimator, R-LINS, is presented in [16]. R-LINS uses an error-state Kalman filter to correct a robot's state estimate

recursively in a tightly-coupled manner. Due to the lack of other available sensors for state estimation, it suffers from drift during long-during navigation. A tightly-coupled lidar inertial odometry and mapping framework, LIOM, is introduced in [17]. LIOM, which is the abbreviation for LIO-mapping, jointly optimizes measurements from lidar and IMU and achieves similar or better accuracy when compared with LOAM. Since LIOM is designed to process all the sensor measurements, real-time performance is not achieved - it runs at about $0.6 \times$ real-time in our tests.

III. LIDAR INERTIAL ODOMETRY VIA SMOOTHING AND MAPPING

A. System Overview

We first define frames and notation that we use throughout the paper. We denote the world frame as $\mathbf{W}$ and the robot body frame as $\mathbf{B}$. We also assume the IMU frame coincides with the robot body frame for convenience. The robot state $\mathbf{x}$ can be written as:

$$\mathbf{x} = [\mathbf{R}^T, \mathbf{p}^T, \mathbf{v}^T, \mathbf{b}^T]^T, \quad (1)$$

where $\mathbf{R} \in SO(3)$ is the rotation matrix, $\mathbf{p} \in \mathbb{R}^3$ is the position vector, $\mathbf{v}$ is the speed, and $\mathbf{b}$ is the IMU bias. The transformation $\mathbf{T} \in SE(3)$ from $\mathbf{B}$ to $\mathbf{W}$ is represented as $\mathbf{T} = [\mathbf{R} \mid \mathbf{p}]$.

An overview of the proposed system is shown in Figure 1. The system receives sensor data from a 3D lidar, an IMU and optionally a GPS. We seek to estimate the state of the robot and its trajectory using the observations of these sensors. This state estimation problem can be formulated as a maximum a posteriori (MAP) problem. We use a factor graph to model this problem, as it is better suited to perform inference when compared with Bayes nets. With the assumption of a Gaussian noise model, the MAP inference for our problem is equivalent to solving a nonlinear least-squares problem [18]. Note that without loss of generality, the proposed system can also incorporate measurements from other sensors, such as elevation from an altimeter or heading from a compass.

We introduce four types of *factors* along with one *variable* type for factor graph construction. This variable, representing the robot's state at a specific time, is attributed

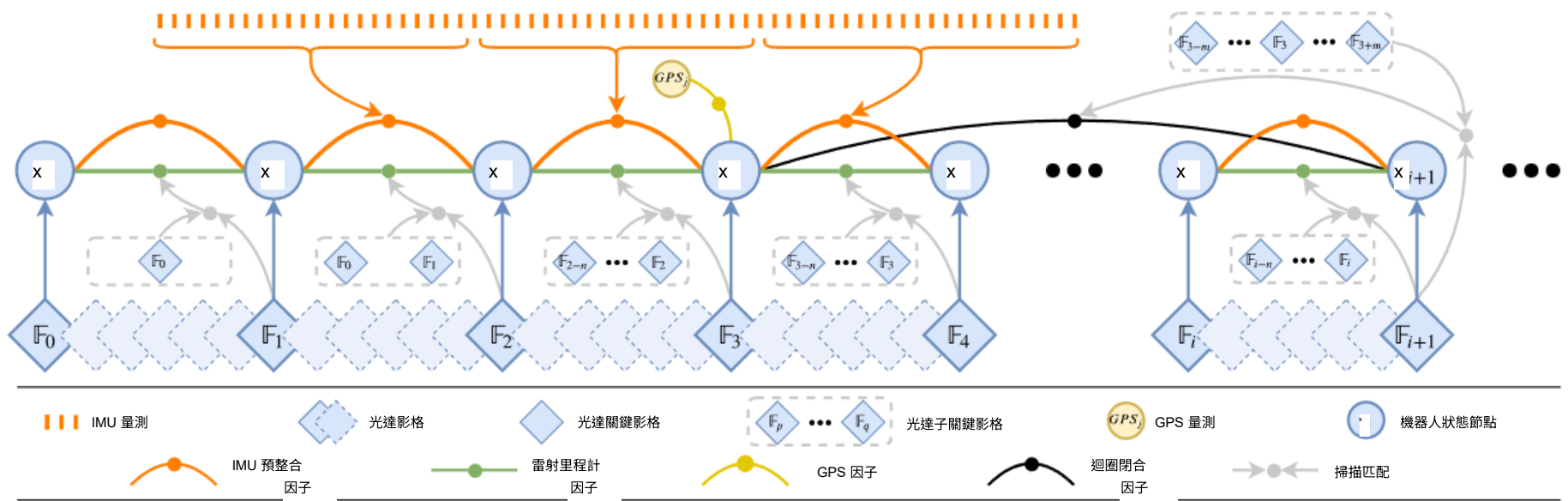


圖 1：LIO-SAM 的系統結構。系統接收來自 3D 光達、IMU 和可選的 GPS 輸入。引入四種類型的因子來建構因子圖：(a) IMU 預整合因子，(b) 光達里程計因子，(c) GPS 因子，以及 (d) 迴路閉合因子。這些因子的生成將在第三節中討論。

二、相關工作

光達里程計通常透過使用掃描匹配方法（例如 ICP [3] 和 GICP [4]）來尋找兩個連續影格之間的相對轉換來執行。基於特徵的匹配方法由於其計算效率，已成為一種流行的替代方案，而不是匹配完整的點雲。例如，在 [5] 中，提出了一種基於平面的配準方法，用於即時光達里程計。假設在結構化環境中操作，它從點雲中提取平面，並透過求解最小平方方法問題來匹配它們。在 [6] 中，提出了一種基於領線的方法用於里程計估計。在此方法中，線段是從原始點雲中隨機生成的，並隨後用於配準。然而，由於現代 3D 光達的旋轉機制和感測器運動，掃描的點雲通常會傾斜。僅使用光達進行姿態估計並不理想，因為使用傾斜點雲或特徵進行配準最終會導致大的漂移。

因此，光達通常會與其他感測器（例如 IMU 和 GPS）結合使用，以進行狀態估計和繪圖。這種利用感測器融合的設計方案，通常可分為兩類：鬆散耦合融合和緊密耦合融合。在 LOAM [1] 中，引入 IMU 以消除光達掃描的傾斜，並為掃描匹配提供運動先驗。然而，IMU 並未參與演算法的優化過程。因此，LOAM 可歸類為鬆散耦合方法。文獻 [7] 提出了一種輕量級且針對地面優化的光達里程計和繪圖 (LeGOLOAM) 方法，用於地面車輛繪圖任務 [8]。其 IMU 量測的融合方式與 LOAM 相同。鬆散耦合融合的一種更流行的方法是使用擴展卡爾曼濾波器 (EKF)。例如，[9][13] 在優化階段使用 EKF 整合來自光達、IMU 和可選 GPS 的量測，以進行機器人狀態估計。

緊密耦合系統通常能提供更高的準確性，並且目前是持續研究的重點 [14]。在 [15] 中，利用預整合的 IMU 量測來消除點雲的傾斜。文獻 [16] 提出了一種以機器人為中心的雷射慣性狀態估計器 R-LINS。

R-LINS 使用誤差狀態卡爾曼濾波器以緊密耦合的方式遞迴校正機器人的狀態估計。由於缺乏其他可用的感測器進行狀態估計，它在長時間導航期間會出現漂移。文獻 [17] 介紹了一種緊密耦合的雷射慣性里程計和繪圖框架 LIOM。LIOM 是 LIO-mapping 的縮寫，它共同優化來自雷射和 IMU 的量測，與 LOAM 相比，實現了相似或更好的準確性。由於 LIOM 旨在處理所有感測器量測，因此無法實現即時性能——在我們的測試中，它以約 0.6 倍的即時速度運行。

SM

A. 系統概述

我們首先定義本文中使用的座標系和符號。我們將世界座標系表示為 W ，機器人本體座標系表示為 B 。為方便起見，我們也假設 IMU 座標系與機器人本體座標系重合。機器人狀態 x 可以寫為：

$$x = [R, p, v, b]^T, \quad (1)$$

其中 $R \in SO(3)$ 是旋轉矩陣， $p \in \mathbb{R}^3$ 是位置向量， v 是速度， b 是 IMU 偏差。從 B 到 W 的變換 $T \in SE(3)$ 表示為 $T = [R | p]$ 。

所提出系統的概述如圖 1 所示。

該系統接收來自 3D 光達、IMU 和可選的 GPS 的感測器資料。我們試圖利用這些感測器的觀測值來估計機器人的狀態及其軌跡。這個狀態估計問題可以表述為最大後驗機率 (MAP) 問題。我們使用因子圖來建模這個問題，因為它比貝葉斯網路更適合執行推斷。在假設高斯雜訊模型的情況下，我們問題的 MAP 推斷等同於求解非線性最小平方方法問題 [18]。請注意，在不失一般性的前提下，所提出的系統還可以整合來自其他感測器的測量值，例如高度計的高度或指南針的航向。

我們引入了四種類型的因子以及一種用於因子圖建構的變數類型。這個變數代表機器人在特定時間的狀態，並被賦予

to the *nodes* of the graph. The four types of factors are: (a) IMU preintegration factors, (b) lidar odometry factors, (c) GPS factors, and (d) loop closure factors. A new robot state node $\mathbf{x}$ is added to the graph when the change in robot pose exceeds a user-defined threshold. The factor graph is optimized upon the insertion of a new node using incremental smoothing and mapping with the Bayes tree (iSAM2) [19]. The process for generating these factors is described in the following sections.

B. IMU Preintegration Factor

The measurements of angular velocity and acceleration from an IMU are defined using Eqs. 2 and 3:

$$\hat{\omega}_t = \omega_t + \mathbf{b}_t^\omega + \mathbf{n}_t^\omega \quad (2)$$

$$\hat{\mathbf{a}}_t = \mathbf{R}_t^{\mathbf{B}\mathbf{W}}(\mathbf{a}_t - \mathbf{g}) + \mathbf{b}_t^{\mathbf{a}} + \mathbf{n}_t^{\mathbf{a}}, \quad (3)$$

where $\hat{\omega}_t$ and $\hat{\mathbf{a}}_t$ are the raw IMU measurements in $\mathbf{B}$ at time t . $\hat{\omega}_t$ and $\hat{\mathbf{a}}_t$ are affected by a slowly varying bias $\mathbf{b}_t$ and white noise $\mathbf{n}_t$. $\mathbf{R}_t^{\mathbf{B}\mathbf{W}}$ is the rotation matrix from $\mathbf{W}$ to $\mathbf{B}$. $\mathbf{g}$ is the constant gravity vector in $\mathbf{W}$.

We can now use the measurements from the IMU to infer the motion of the robot. The velocity, position and rotation of the robot at time $t + \Delta t$ can be computed as follows:

$$\mathbf{v}_{t+\Delta t} = \mathbf{v}_t + \mathbf{g}\Delta t + \mathbf{R}_t(\hat{\mathbf{a}}_t - \mathbf{b}_t^{\mathbf{a}} - \mathbf{n}_t^{\mathbf{a}})\Delta t \quad (4)$$

$$\begin{aligned} \mathbf{p}_{t+\Delta t} = \mathbf{p}_t + \mathbf{v}_t\Delta t + \frac{1}{2}\mathbf{g}\Delta t^2 \\ + \frac{1}{2}\mathbf{R}_t(\hat{\mathbf{a}}_t - \mathbf{b}_t^{\mathbf{a}} - \mathbf{n}_t^{\mathbf{a}})\Delta t^2 \end{aligned} \quad (5)$$

$$\mathbf{R}_{t+\Delta t} = \mathbf{R}_t \exp((\hat{\omega}_t - \mathbf{b}_t^\omega - \mathbf{n}_t^\omega)\Delta t), \quad (6)$$

where $\mathbf{R}_t = \mathbf{R}_t^{\mathbf{W}\mathbf{B}} = \mathbf{R}_t^{\mathbf{B}\mathbf{W}\top}$. Here we assume that the angular velocity and the acceleration of $\mathbf{B}$ remain constant during the above integration.

We then apply the IMU preintegration method proposed in [20] to obtain the relative body motion between two timesteps. The preintegrated measurements $\Delta\mathbf{v}_{ij}$, $\Delta\mathbf{p}_{ij}$, and $\Delta\mathbf{R}_{ij}$ between time i and j can be computed using:

$$\Delta\mathbf{v}_{ij} = \mathbf{R}_i^\top(\mathbf{v}_j - \mathbf{v}_i - \mathbf{g}\Delta t_{ij}) \quad (7)$$

$$\Delta\mathbf{p}_{ij} = \mathbf{R}_i^\top(\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i\Delta t_{ij} - \frac{1}{2}\mathbf{g}\Delta t_{ij}^2) \quad (8)$$

$$\Delta\mathbf{R}_{ij} = \mathbf{R}_i^\top \mathbf{R}_j. \quad (9)$$

Due to space limitations, we refer the reader to the description from [20] for the detailed derivation of Eqs. 7 - 9. Besides its efficiency, applying IMU preintegration also naturally gives us one type of constraint for the factor graph - IMU preintegration factors. The IMU bias is jointly optimized alongside the lidar odometry factors in the graph.

C. Lidar Odometry Factor

When a new lidar scan arrives, we first perform feature extraction. Edge and planar features are extracted by evaluating the roughness of points over a local region. Points with a large roughness value are classified as edge features. Similarly, a planar feature is categorized by a small roughness value. We denote the extracted edge and planar

features from a lidar scan at time i as F_i^e and F_i^p respectively. All the features extracted at time i compose a lidar *frame* $\mathbb{F}_i$, where $\mathbb{F}_i = \{F_i^e, F_i^p\}$. Note that a lidar frame $\mathbb{F}$ is represented in $\mathbf{B}$. A more detailed description of the feature extraction process can be found in [1], or [7] if a range image is used.

Using every lidar frame for computing and adding factors to the graph is computationally intractable, so we adopt the concept of *keyframe* selection, which is widely used in the visual SLAM field. Using a simple but effective heuristic approach, we select a lidar frame $\mathbb{F}_{i+1}$ as a keyframe when the change in robot pose exceeds a user-defined threshold when compared with the previous state $\mathbf{x}_i$. The newly saved keyframe, $\mathbb{F}_{i+1}$, is associated with a new robot state node, $\mathbf{x}_{i+1}$, in the factor graph. The lidar frames between two keyframes are discarded. Adding keyframes in this way not only achieves a balance between map density and memory consumption but also helps maintain a relatively sparse factor graph, which is suitable for real-time nonlinear optimization. In our work, the position and rotation change thresholds for adding a new keyframe are chosen to be $1m$ and 10° .

Let us assume we wish to add a new state node $\mathbf{x}_{i+1}$ to the factor graph. The lidar keyframe that is associated with this state is $\mathbb{F}_{i+1}$. The generation of a lidar odometry factor is described in the following steps:

1) *Sub-keyframes for voxel map*: We implement a sliding window approach to create a point cloud map containing a fixed number of recent lidar scans. Instead of optimizing the transformation between two consecutive lidar scans, we extract the n most recent keyframes, which we call the *sub-keyframes*, for estimation. The set of sub-keyframes $\{\mathbb{F}_{i-n}, \dots, \mathbb{F}_i\}$ is then transformed into frame $\mathbf{W}$ using the transformations $\{\mathbf{T}_{i-n}, \dots, \mathbf{T}_i\}$ associated with them. The transformed sub-keyframes are merged together into a voxel map $\mathbf{M}_i$. Since we extract two types of features in the previous feature extraction step, $\mathbf{M}_i$ is composed of two sub-voxel maps that are denoted $\mathbf{M}_i^e$, the edge feature voxel map, and $\mathbf{M}_i^p$, the planar feature voxel map. The lidar frames and voxel maps are related to each other as follows:

$$\mathbf{M}_i = \{\mathbf{M}_i^e, \mathbf{M}_i^p\}$$

$$\text{where : } \mathbf{M}_i^e = 'F_i^e \cup F_{i-1}^e \cup \dots \cup F_{i-n}^e$$

$$\mathbf{M}_i^p = 'F_i^p \cup F_{i-1}^p \cup \dots \cup F_{i-n}^p.$$

' F_i^e and ' F_i^p are the transformed edge and planar features in $\mathbf{W}$. $\mathbf{M}_i^e$ and $\mathbf{M}_i^p$ are then downsampled to eliminate the duplicated features that fall in the same voxel cell. In this paper, n is chosen to be 25. The downsample resolutions for $\mathbf{M}_i^e$ and $\mathbf{M}_i^p$ are $0.2m$ and $0.4m$, respectively.

2) *Scan-matching*: We match a newly obtained lidar frame $\mathbb{F}_{i+1}$, which is also $\{F_{i+1}^e, F_{i+1}^p\}$, to $\mathbf{M}_i$ via scan-matching. Various scan-matching methods, such as [3] and [4], can be utilized for this purpose. Here we opt to use the method proposed in [1] due to its computational efficiency and robustness in various challenging environments.

We first transform $\{F_{i+1}^e, F_{i+1}^p\}$ from $\mathbf{B}$ to $\mathbf{W}$ and obtain $\{F_{i+1}^e, F_{i+1}^p\}$. This initial transformation is obtained by using the predicted robot motion, $\tilde{\mathbf{T}}_{i+1}$, from the IMU. For



到圖的節點。這四種類型的因子是：(a) IMU 預整合因子、(b) 雷射雷達里程計因子、(c) GPS 因子，以及 (d) 閉迴路因子。當機器人姿態的變化超過使用者定義的閾值時，新的機器人狀態節點 x 會被新增到圖中。因子圖在插入新節點時，會使用貝葉斯樹的增量平滑與映射 (iSAM2) [19] 進行優化。產生這些因子的過程將在以下各節中說明。

B. IMU 預積分因子

IMU 的角速度和加速度測量值使用方程式 2 和 3 定義：

$$\hat{\omega} = \omega + b + n \quad (2)$$

$$\hat{a} = R^T(a - g) + b + n, \quad (3)$$

其中 $\hat{\omega}$ 和 $\hat{a}$ 是時間 t 在 B 中的原始 IMU 測量值。 $\hat{\omega}$ 和 $\hat{a}$ 會受到緩慢變化的偏差 b 和白雜訊 n 的影響。 R

是從 W 到 B 的旋轉矩陣。 g 是 W 中的常數重力向量。

我們現在可以使用 IMU 的測量值來推斷機器人的運動。機器人在時間 $t + \Delta t$ 的速度、位置和旋轉可以計算如下：

$$v = v + g\Delta t + R(\hat{a} - b - n)\Delta t \quad (4)$$

$$p = p + v\Delta t + \frac{1}{2}g\Delta t^2 + \frac{1}{2}R(\hat{a} - b - n)\Delta t^2 \quad (5)$$

$$R = R \exp((\hat{\omega} - b - n)\Delta t), \quad (6)$$

其這裡我們假設在上述積分過程中， B 的角速度和加速度保持不變。

然後，我們應用 [20] 中提出的 IMU 預積分方法，以獲得兩個時間步之間的相對物體運動。時間 i 和 j 之間的預積分測量 Δv 、 Δp 和 ΔR 可以使用以下方式計算：

$$\Delta v = R(v - v - g\Delta t) \quad (7)$$

$$\Delta p = R(p - p - v\Delta t - \frac{1}{2}g\Delta t^2) \quad (8)$$

$$\Delta R = R R. \quad (9)$$

由於空間限制，詳細的方程式 7-9 推導請參閱文獻 [20] 的說明。除了效率之外，應用 IMU 預積分也自然地為我們提供了一種因子圖約束——IMU 預積分因子。IMU 偏差與圖中的雷射雷達里程計因子共同最佳化。

C. 雷射雷達里程計因子

當新的雷射雷達掃描資料到達時，我們首先執行特徵提取。透過評估局部區域點的粗糙度來提取邊緣和平面特徵。粗糙度值大的點被歸類為邊緣特徵。同樣地，平面特徵則由小的粗糙度值來分類。

我們將在時間 i 從雷射雷達掃描中提取的邊緣和平面特徵分別表示為 F 和 F 。在時間 i 提取的所有特徵組成一個雷射雷達幀 F ，其中 $F = \{F, F\}$ 。請注意，雷射雷達幀 F 以 B 表示。有關特徵提取過程的更詳細描述，請參閱 [1]，如果使用距離影像，則請參閱 [7]。

由於使用每個光達影格來計算並將因子新增至圖中，在計算上是不可行的，因此我們採用了在視覺 SLAM 領域中廣泛使用的關鍵影格選擇概念。我們使用一種簡單但有效的啟發式方法，當機器人姿態的變化與先前的狀態 x 相比超過使用者定義的閾值時，我們選擇光達影格 F 作為關鍵影格。新儲存的關鍵影格 F 會在因子圖中與新的機器人狀態節點 x 相關聯。兩個關鍵影格之間的光達影格會被捨棄。以這種方式新增關鍵影格不僅能在地圖密度和記憶體消耗之間取得平衡，還有助於維持相對稀疏的因子圖，這適合即時非線性最佳化。在我們的工作中，新增新關鍵影格的位置和旋轉變化閾值選擇為 1 公尺和 10 度。

讓我們假設我們希望將新的狀態節點 x 新增至因子圖中。與此狀態相關聯的光達關鍵影格為 F 。光達里程計因子的產生步驟如下：

1) 用於體素地圖的子關鍵影格：我們實作了一個滑動視窗方法來建立一個包含固定數量近期光達掃描的點雲地圖。我們不最佳化兩個連續光達掃描之間的轉換，而是提取最近的 n 個關鍵影格（我們稱之為子關鍵影格）進行估計。然後，子關鍵影格集 $\{F, \dots, F\}$ 會使用與其相關聯的轉換 $\{T, \dots, T\}$ 轉換為影格 W 。轉換後的子關鍵影格會合併成一個體素地圖 M 。由於我們在先前的特徵提取步驟中提取了兩種特徵，因此 M 由兩個子體素地圖組成，分別表示為邊緣特徵體素地圖 M 和平面特徵體素地圖 M 。光達影格和體素地圖之間的關係如下：

$$M = \{M, M\}$$

$$\text{其中：} M = F U F U \dots U F$$

$$M = F U F U \dots U F。$$

F 和 F 是 W 中轉換後的邊緣和平面特徵。 M 和 M 隨後被降採樣，以消除落在相同體素單元中的重複特徵。在本文中， n 選擇為 25。 M 和 M 的降採樣解析度分別為 0.2m 和 0.4m。

2) 掃描匹配：我們將新獲得的雷射雷達影格 F （也是 $\{F, F\}$ ）透過掃描匹配與 M 進行匹配。為此，可以使用各種掃描匹配方法，例如 [3] 和 [4]。在這裡，我們選擇使用 [1] 中提出的方法，因為它在各種挑戰性環境中具有計算效率和穩健性。

我們首先將 $\{F, F\}$ 從 B 轉換為 W ，並獲得 $\{F, F\}$ 。這個初始轉換是透過使用 IMU 預測的機器人運動 $\sim T$ 獲得的。對於

each feature in $'F_{i+1}^e$ or $'F_{i+1}^p$, we then find its edge or planar correspondence in M_i^e or M_i^p . For the sake of brevity, the detailed procedures for finding these correspondences are omitted here, but are described thoroughly in [1].

3) *Relative transformation*: The distance between a feature and its edge or planar patch correspondence can be computed using the following equations:

$$d_{e_k} = \frac{|(\mathbf{p}_{i+1,k}^e - \mathbf{p}_{i,u}^e) \times (\mathbf{p}_{i+1,k}^e - \mathbf{p}_{i,v}^e)|}{|\mathbf{p}_{i,u}^e - \mathbf{p}_{i,v}^e|} \quad (10)$$

$$d_{p_k} = \frac{|(\mathbf{p}_{i+1,k}^p - \mathbf{p}_{i,u}^p) \times (\mathbf{p}_{i+1,k}^p - \mathbf{p}_{i,w}^p)|}{|(\mathbf{p}_{i,u}^p - \mathbf{p}_{i,v}^p) \times (\mathbf{p}_{i,u}^p - \mathbf{p}_{i,w}^p)|}, \quad (11)$$

where k , u , v , and w are the feature indices in their corresponding sets. For an edge feature $\mathbf{p}_{i+1,k}^e$ in $'F_{i+1}^e$, $\mathbf{p}_{i,u}^e$ and $\mathbf{p}_{i,v}^e$ are the points that form the corresponding edge line in M_i^e . For a planar feature $\mathbf{p}_{i+1,k}^p$ in $'F_{i+1}^p$, $\mathbf{p}_{i,u}^p$, $\mathbf{p}_{i,v}^p$, and $\mathbf{p}_{i,w}^p$ form the corresponding planar patch in M_i^p . The GaussNewton method is then used to solve for the optimal transformation by minimizing:

$$\min_{\mathbf{T}_{i+1}} \left\{ \sum_{\mathbf{p}_{i+1,k}^e \in 'F_{i+1}^e} d_{e_k} + \sum_{\mathbf{p}_{i+1,k}^p \in 'F_{i+1}^p} d_{p_k} \right\}.$$

At last, we can obtain the relative transformation $\Delta\mathbf{T}_{i,i+1}$ between $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$, which is the lidar odometry factor linking these two poses:

$$\Delta\mathbf{T}_{i,i+1} = \mathbf{T}_i^T \mathbf{T}_{i+1} \quad (12)$$

We note that an alternative approach to obtain $\Delta\mathbf{T}_{i,i+1}$ is to transform sub-keyframes into the frame of $\mathbf{x}_i$. In other words, we match $\mathbb{F}_{i+1}$ to the voxel map that is represented in the frame of $\mathbf{x}_i$. In this way, the *real* relative transformation $\Delta\mathbf{T}_{i,i+1}$ can be directly obtained. Because the transformed features $'F_i^e$ and $'F_i^p$ can be reused multiple times, we instead opt to use the approach described in Sec. III-C.1 for its computational efficiency.

D. GPS Factor

Though we can obtain reliable state estimation and mapping by utilizing only IMU preintegration and lidar odometry factors, the system still suffers from drift during long-duration navigation tasks. To solve this problem, we can introduce sensors that offer absolute measurements for eliminating drift. Such sensors include an altimeter, compass, and GPS. For the purposes of illustration here, we discuss GPS, as it is widely used in real-world navigation systems.

When we receive GPS measurements, we first transform them to the local Cartesian coordinate frame using the method proposed in [21]. Upon the addition of a new node to the factor graph, we then associate a new GPS factor with this node. If the GPS signal is not hardware-synchronized with the lidar frame, we interpolate among GPS measurements linearly based on the timestamp of the lidar frame.



(a) Handheld device (b) Clearpath Jackal (c) Duffy 21

Fig. 2: Datasets are collected on 3 platforms: (a) a custom-built handheld device, (b) an unmanned ground vehicle - Clearpath Jackal, (c) an electric boat - Duffy 21.

We note that adding GPS factors constantly when GPS reception is available is not necessary because the drift of lidar inertial odometry grows very slowly. In practice, we only add a GPS factor when the estimated position covariance is larger than the received GPS position covariance.

E. Loop Closure Factor

Thanks to the utilization of a factor graph, loop closures can also be seamlessly incorporated into the proposed system, as opposed to LOAM and LIOM. For the purposes of illustration, we describe and implement a naive but effective Euclidean distance-based loop closure detection approach. We also note that our proposed framework is compatible with other methods for loop closure detection, for example, [22] and [23], which generate a point cloud descriptor and use it for place recognition.

When a new state $\mathbf{x}_{i+1}$ is added to the factor graph, we first search the graph and find the prior states that are close to $\mathbf{x}_{i+1}$ in Euclidean space. As is shown in Fig. 1, for example, $\mathbf{x}_3$ is one of the returned candidates. We then try to match $\mathbb{F}_{i+1}$ to the sub-keyframes $\{\mathbb{F}_{3-m}, \dots, \mathbb{F}_3, \dots, \mathbb{F}_{3+m}\}$ using scan-matching. Note that $\mathbb{F}_{i+1}$ and the past sub-keyframes are first transformed into $\mathbf{W}$ before scan-matching. We obtain the relative transformation $\Delta\mathbf{T}_{3,i+1}$ and add it as a loop closure factor to the graph. Throughout this paper, we choose the index m to be 12, and the search distance for loop closures is set to be $15m$ from a new state $\mathbf{x}_{i+1}$.

In practice, we find adding loop closure factors is especially useful for correcting the drift in a robot's altitude, when GPS is the only absolute sensor available. This is because the elevation measurement from GPS is very inaccurate - giving rise to altitude errors approaching $100m$ in our tests, in the absence of loop closures.

IV. EXPERIMENTS

We now describe a series of experiments to qualitatively and quantitatively analyze our proposed framework. The sensor suite used in this paper includes a Velodyne VLP-16 lidar, a MicroStrain 3DM-GX5-25 IMU, and a Reach M GPS. For validation, we collected 5 different datasets across various scales, platforms and environments. These datasets are referred to as *Rotation*, *Walking*, *Campus*, *Park* and *Amsterdam*, respectively. The sensor mounting platforms are shown in Fig. 2. The first three datasets were collected using a custom-built handheld device on the MIT campus. The Park

對於 F 中的每個特徵，我們接著在 M 中找到其邊緣或平面對應。為求簡潔，此處省略了尋找這些對應的詳細步驟，但其在 [1] 中有詳盡描述。

3) 相對轉換：特徵之間的距離
構圖及其邊緣或平面塊對應關係可使用以下方程式計算：

$$d = \frac{|(p - p) \times (p - p)|}{|p - p|} \quad (10)$$

$$d = \frac{\left| \frac{(p - p)}{(p - p) \times (p - p)} \right|}{\left| \frac{(p - p) \times (p - p)}{(p - p) \times (p - p)} \right|}, \quad (11)$$

其中 k 、 u 、 v 和 w 是其對應集合中的特徵索引。對於邊緣特徵 $p \in F$ ， p 和 p 是在 M 中形成對應邊緣線的點。對於平面特徵 $p \in F$ ， p 、 p 和 p 在 M 中形成對應的平面塊。然後使用高斯牛頓方法透過最小化以下式子來求解最佳轉換：

$$\min_T \left\{ \sum_{p \in F} d + \sum_{p \in F} d \right\}.$$

最後，我們可以得到 x 與 x 之間的相對轉換 ΔT ，這是連結這兩個姿態的雷射雷達里程計因子：

$$\Delta T = T T \quad (12)$$

我們注意到，取得 ΔT 的另一種方法是將子關鍵影格轉換為 x 的影格。換句話說，我們將 F 與以 x 影格表示的體素地圖進行匹配。透過這種方式，可以直接取得真實的相對轉換 ΔT 。由於轉換後的特徵 F 和 F 可以重複使用多次，因此我們選擇使用第 III-C.1 節中描述的方法，以提高其計算效率。

D. GPS 因子

儘管我們僅利用 IMU 預積分和光達里程計因子即可獲得可靠的狀態估計和地圖繪製，但系統在長時間導航任務中仍會出現漂移。為了解決這個問題，我們可以引入提供絕對測量的感測器來消除漂移。這類感測器包括高度計、指南針和 GPS。為了在此處進行說明，我們討論 GPS，因為它在現實世界的導航系統中廣泛使用。

當我們收到 GPS 測量值時，我們首先使用 [21] 中提出的方法將其轉換為局部笛卡爾座標系。在因子圖中新增節點後，我們將新的 GPS 因子與此節點關聯。如果 GPS 訊號未與光達影格進行硬體同步，我們則根據光達影格的時間戳記在 GPS 測量值之間進行線性插值。



(a) 手持裝置

(b) Clearpath Jackal

(c) Duffy 21

圖 2：資料集是在 3 個平台上收集的：(a)客製化手持裝置、(b)無人地面載具 Clearpath Jackal、(c)電動船 Duffy 21。

我們注意到，當 GPS 可用時，持續添加 GPS 因子並非必要，因為光達慣性里程計的漂移增長非常緩慢。實際上，我們只在估計位置協方差大於接收到的 GPS 位置協方差時才添加 GPS 因子。

E. 迴路閉合因子

由於使用了因子圖，迴路閉合也可以無縫地整合到所提出的系統中，這與 LOAM 和 LIOM 不同。為了說明目的，我們描述並實施了一種簡單但有效的基於歐幾里得距離的迴路閉合檢測方法。我們還注意到，我們提出的框架與其他迴路閉合檢測方法相容，例如 [22] 和 [23]，它們生成點雲描述符並將其用於地點識別。

當新的狀態 x 被添加到因子圖時，我們首先搜尋圖並找到在歐幾里得空間中接近 x 的先前狀態。如圖 1 所示，例如， x 是其中一個返回的候選。然後，我們嘗試使用掃描匹配將 F 與子關鍵幀 $\{F, \dots, F, \dots, F\}$ 進行匹配。請注意， F 和過去的子關鍵幀在掃描匹配之前會先轉換為 W 。我們獲得相對變換 ΔT 並將其作為迴路閉合因子添加到圖中。在本文中，我們選擇索引 m 為 12，並且迴路閉合的搜尋距離設定為距離新狀態 x 15 公尺。

在實務上，我們發現當 GPS 是唯一可用的絕對感測器時，新增閉環因子對於校正機器人高度的漂移特別有用。這是因為 GPS 的高程測量非常不準確，在沒有閉環的情況下，我們的測試中會產生接近 100 公尺的高度誤差。

IV. E

我們現在將描述一系列實驗，以定性和定量分析我們提出的框架。本文使用的感測器套件包括 Velodyne VLP-16 光達、MicroStrain 3DM-GX5-25 IMU 和 Reach M GPS。為了驗證，我們收集了 5 個不同規模、平台和環境的資料集。這些資料集分別稱為 Rotation、Walking、Campus、Park 和 Amsterdam。感測器安裝平台如圖 2 所示。前三個資料集是使用麻省理工學院校園內客製化的手持裝置收集的。Park

dataset was collected in a park covered by vegetation, using an unmanned ground vehicle (UGV) - the Clearpath Jackal. The last dataset, Amsterdam, was collected by mounting the sensors on a boat and cruising in the canals of Amsterdam. The details of these datasets are shown in Table I.

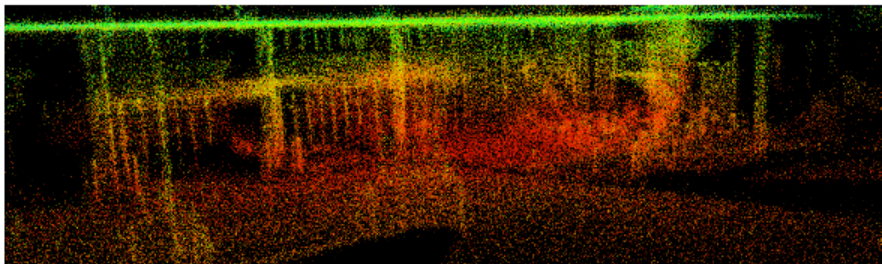
TABLE I: Dataset details

Dataset	Scans	Elevation change (m)	Trajectory length (m)	Max rotation speed ($^{\circ}/s$)
Rotation	582	0	0	213.9
Walking	6502	0.3	801	133.7
Campus	9865	1.0	1437	124.8
Park	24691	19.0	2898	217.4
Amsterdam	107656	0	19065	17.2

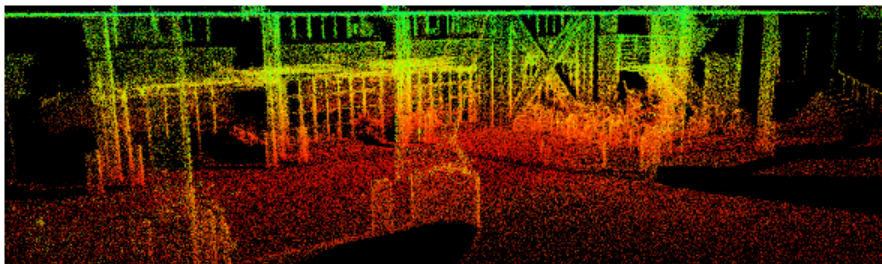
We compare the proposed LIO-SAM framework with LOAM and LIOM. In all the experiments, LOAM and LIO-SAM are forced to run in real-time. LIOM, on the other hand, is given infinite time to process every sensor measurement. All the methods are implemented in C++ and executed on a laptop equipped with an Intel i7-10710U CPU using the robot operating system (ROS) [24] in Ubuntu Linux. We note that only the CPU is used for computation, without parallel computing enabled. Our implementation of LIO-SAM is freely available on Github¹. Supplementary details of the experiments performed, including complete visualizations of all tests, can be found at the link below².



(a) Test environment



(b) LOAM



(c) LIO-SAM

Fig. 3: Mapping results of LOAM and LIO-SAM in the *Rotation* test. LIOM fails to produce meaningful results.

A. Rotation Dataset

In this test, we focus on evaluating the robustness of our framework when only IMU preintegration and lidar odometry

¹<https://github.com/TixiaoShan/LIO-SAM>

²<https://youtu.be/A0H8CoORZJU>

factors are added to the factor graph. The *Rotation* dataset is collected by a user holding the sensor suite and performing a series of aggressive rotational maneuvers while standing still. The maximum rotational speed encountered in this test is $133.7^{\circ}/s$. The test environment, which is populated with structures, is shown in Fig. 3(a). The maps obtained from LOAM and LIO-SAM are shown in Figs. 3(b) and (c) respectively. Because LIOM uses the same initialization pipeline from [25], it inherits the same initialization sensitivity of visual-inertial SLAM and is not able to initialize properly using this dataset. Due to its failure to produce meaningful results, the map of LIOM is not shown. As is shown, the map of LIO-SAM preserves more fine structural details of the environment compared with the map of LOAM. This is because LIO-SAM is able to register each lidar frame precisely in $SO(3)$, even when the robot undergoes rapid rotation.

B. Walking Dataset

This test is designed to evaluate the performance of our method when the system undergoes aggressive translations and rotations in $SE(3)$. The maximum translational and rotational speed encountered in this dataset is 1.8 m/s and $213.9^{\circ}/s$ respectively. During the data gathering, the user holds the sensor suite shown in Fig. 2(a) and walks quickly across the MIT campus (Fig. 4(a)). In this test, the map of LOAM, shown in Fig. 4(b), diverges at multiple locations when aggressive rotation is encountered. LIOM outperforms LOAM in this test. However, its map, shown in Fig. 4(c), still diverges slightly in various locations and consists of numerous blurry structures. Because LIOM is designed to process all sensor measurements, it only runs at $0.56\times$ real-time while other methods are running in real-time. Finally, LIO-SAM outperforms both methods and produces a map that is consistent with the available Google Earth imagery.

C. Campus Dataset

TABLE II: End-to-end translation error (meters)

Dataset	LOAM	LIOM	LIO-odom	LIO-GPS	LIO-SAM
Campus	192.43	Fail	9.44	6.87	0.12
Park	121.74	34.60	36.36	2.93	0.04
Amsterdam	Fail	Fail	Fail	1.21	0.17

This test is designed to show the benefits of introducing GPS and loop closure factors. In order to do this, we purposely disable the insertion of GPS and loop closure factors into the graph. When both GPS and loop closure factors are disabled, our method is referred to as *LIO-odom*, which only utilizes IMU preintegration and lidar odometry factors. When GPS factors are used, our method is referred to as *LIO-GPS*, which uses IMU preintegration, lidar odometry, and GPS factors for graph construction. LIO-SAM uses all factors when they are available.

To gather this dataset, the user walks around the MIT campus using the handheld device and returns to the same position. Because of the numerous buildings and trees in the



platforms 和 environments 資料集是在一個植被覆蓋的公園中，使用無人地面車輛 (UGV) - Clearpath Jackal 收集的。最後一個資料集 Amsterdam 是透過將感測器安裝在船上，並在阿姆斯特丹的運河中巡航收集的。這些資料集的詳細資訊如表一所示。

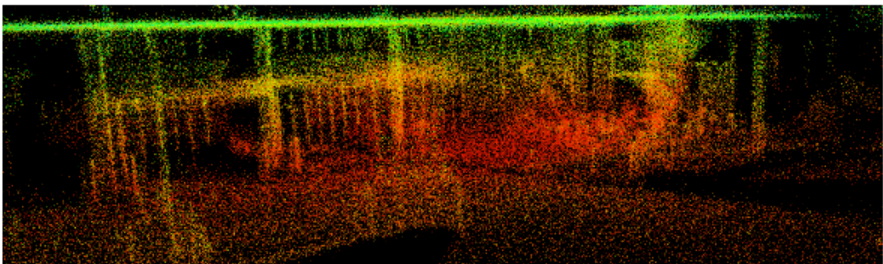
表一：資料集詳細資訊

資料集	掃描次數	海拔變化 (m)	軌跡長度 (m)	最大旋轉速度 (/s)
旋轉	582	0	0	213.9
行走	6502	0.3	801	133.7
校園	9865	1.0	1437	124.8
公園	24691	19.0	2898	217.4
阿姆斯特丹	107656	0	19065	17.2

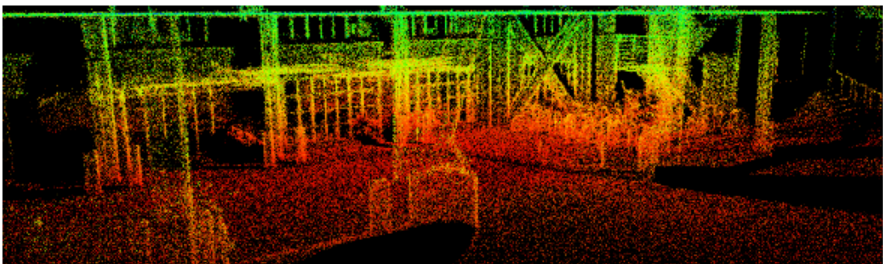
我們將所提出的 LIO-SAM 框架與 LOAM 和 LIOM 進行比較。在所有實驗中，LOAM 和 LIO-SAM 都被迫即時運行。另一方面，LIOM 則有無限的時間來處理每個感測器測量。所有方法都以 C++ 實作，並在配備 Intel i7-10710U CPU 的筆記型電腦上執行，使用 Ubuntu Linux 中的機器人作業系統 (ROS) [24]。我們注意到，僅使用 CPU 進行計算，未啟用平行運算。我們實作的 LIO-SAM 可在 Github 上免費取得。有關所執行實驗的補充詳細資訊，包括所有測試的完整視覺化，請參閱以下連結。



(a) 測試環境



(b) LOAM



(c) LIO-SAM

圖 3：LOAM 和 LIO-SAM 在旋轉測試中的映射結果。LIOM 無法產生有意義的結果。

A. 旋轉資料集

在此測試中，我們專注於評估當僅有 IMU 預整合和雷射雷達里程計時，我們框架的穩健性

因素會被加入到因子圖中。旋轉資料集是由使用者手持感測器套件，在靜止狀態下執行一系列劇烈旋轉操作所收集。此測試中遇到的最大旋轉速度為 133.7 /s。測試環境（其中包含結構）如圖 3(a) 所示。從 LOAM 和 LIO-SAM 獲得的地圖分別如圖 3(b) 和 (c) 所示。由於 LIOM 使用與 [25] 相同的初始化流程，它繼承了視覺慣性 SLAM 相同的初始化敏感性，因此無法使用此資料集正確初始化。由於未能產生有意義的結果，因此未顯示 LIOM 的地圖。如圖所示，與 LOAM 的地圖相比，LIO-SAM 的地圖保留了更多環境的精細結構細節。這是因為 LIO-SAM 即使在機器人快速旋轉時，也能在 SO(3) 中精確地註冊每個雷射雷達幀。

B. 行走資料集

此測試旨在評估當系統在 SE(3) 中經歷劇烈平移和旋轉時，我們方法的效能。此資料集中遇到的最大平移和旋轉速度分別為 1.8 m/s 和

在資料收集期間，使用者手持圖 2(a) 所示的感測器套件，並快速穿過麻省理工學院校園（圖 4(a)）。在此測試中，當遇到劇烈旋轉時，圖 4(b) 所示的 LOAM 地圖在多個位置發散。LIOM 在此測試中優於 LOAM。然而，其地圖（圖 4(c)）在各個位置仍然略微發散，並包含許多模糊的結構。由於 LIOM 旨在處理所有感測器測量值，因此它僅以 0.56 倍的即時速度運行，而其他方法則以即時速度運行。最後，LIO-SAM 優於這兩種方法，並產生與現有 Google Earth 影像一致的地圖。

C. 校園資料集

表二：端到端平移誤差（公尺）

Dataset	LOAM	LIOM	LIO-odom	LIO-GPS	LIO-SAM
Campus	192.43	Fail	9.44	6.87	0.12
Park	121.74	34.60	36.36	2.93	0.04
Amsterdam	Fail	失敗	失敗	1.21	0.17

此測試旨在展示導入 GPS 和迴圈閉合因子的優勢。為此，我們特意停用了將 GPS 和迴圈閉合因子插入圖中。當 GPS 和迴圈閉合因子都停用時，我們的方法稱為 LIO-odom，它僅利用 IMU 預整合和光達里程計因子。當使用 GPS 因子時，我們的方法稱為 LIO-GPS，它使用 IMU 預整合、光達里程計和 GPS 因子來建構圖。LIO-SAM 在可用時會使用所有因子。為了收集此資料集，使用者手持裝置在麻省理工學院校園內走動並返回相同位置。由於校園內有許多建築物和樹木，

¹ <https://github.com/TixiaoShan/LIO-SAM>

² <https://youtu.be/A0H8Co0RZJU>

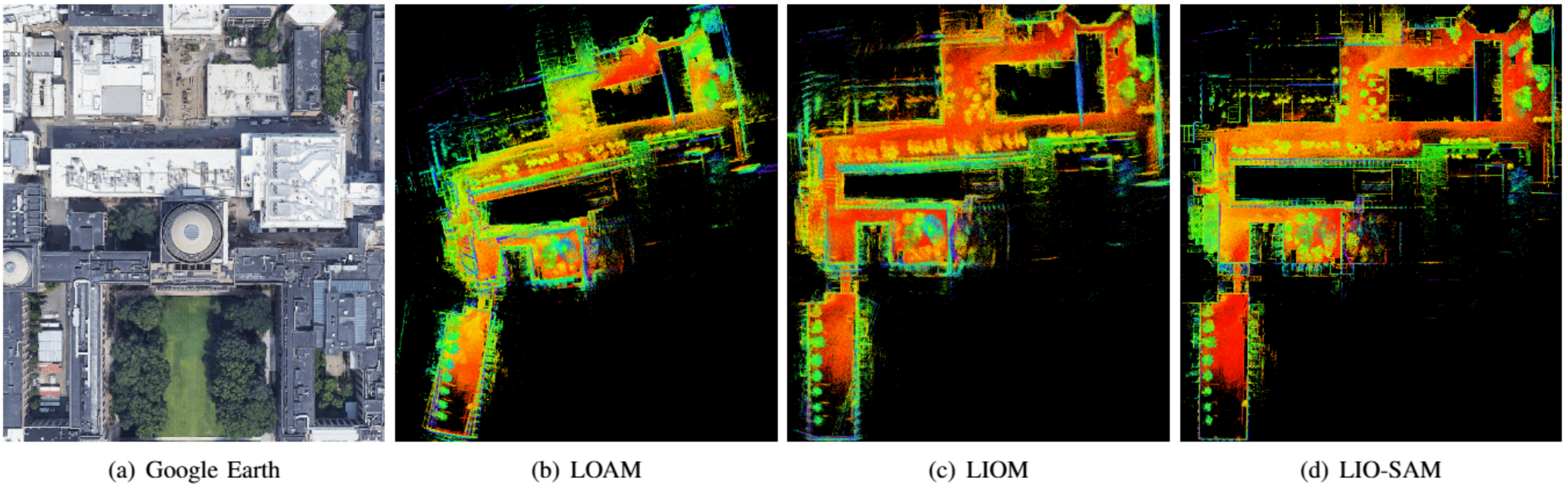


Fig. 4: Mapping results of LOAM, LIOM, and LIO-SAM using the *Walking* dataset. The map of LOAM in (b) diverges multiple times when aggressive rotation is encountered. LIOM outperforms LOAM. However, its map shows numerous blurry structures due to inaccurate point cloud registration. LIO-SAM produces a map that is consistent with the Google Earth imagery, without using GPS.

mapping area, GPS reception is rarely available and inaccurate most of the time. After filtering out the inconsistent GPS measurements, the regions where GPS is available are shown in Fig. 5(a) as green segments. These regions correspond to the few areas that are not surrounded by buildings or trees.

The estimated trajectories of LOAM, LIO-odom, LIO-GPS, and LIO-SAM are shown in Fig. 5(a). The results of LIOM are not shown due to its failure to initialize properly and produce meaningful results. As is shown, the trajectory of LOAM drifts significantly when compared with all other methods. Without the correction of GPS data, the trajectory of LIO-odom begins to visibly drift at the lower right corner of the map. With the help of GPS data, LIO-GPS can correct the drift when it is available. However, GPS data is not available in the later part of the dataset. As a result, LIO-GPS is unable to close the loop when the robot returns to the start position due to drift. On the other hand, LIO-SAM can eliminate the drift by adding loop closure factors to the graph. The map of LIO-SAM is well-aligned with Google Earth imagery and shown in Fig. 5(b). The relative translational error of all methods when the robot returns to the start is shown in Table II.

D. Park Dataset

In this test, we mount the sensors on a UGV and drive the vehicle along a forested hiking trail. The robot returns to its initial position after 40 minutes of driving. The UGV is driven on three road surfaces: asphalt, ground covered by grass, and dirt-covered trails. Due to its lack of suspension, the robot undergoes low amplitude but high frequency vibrations when driven on non-asphalt roads.

To mimic a challenging mapping scenario, we only use GPS measurements when the robot is in widely open areas, which is indicated by the green segments in Fig. 6(a). Such a mapping scenario is representative of a task in which a robot must map multiple GPS-denied regions and periodically returns to regions with GPS availability to correct the drift.

Similar to the results in the previous tests, LOAM, LIOM, and LIO-odom suffer from significant drift, since no absolute correction data is available. Additionally, LIOM only runs at

$0.67\times$ real-time, while the other methods run in real-time. Though the trajectories of LIO-GPS and LIO-SAM coincide in the horizontal plane, their relative translational errors are different (Table II). Because no reliable absolute elevation measurements are available, LIO-GPS suffers from drift in altitude and is unable to close the loop when returning to the robot's initial position. LIO-SAM has no such problem, as it utilizes loop closure factors to eliminate the drift.

E. Amsterdam Dataset

Finally, we mounted the sensor suite on a boat and cruised along the canals of Amsterdam for 3 hours. Although the movement of the sensors is relatively smooth in this test, mapping the canals is still challenging for several reasons. Many bridges over the canals pose degenerate scenarios, as there are few useful features when the boat is under them, similar to moving through a long, featureless corridor. The number of planar features is also significantly less, as the ground is not present. We observe many false detections from the lidar when direct sunlight is in the sensor field-of-view, which occurs about 20% of the time during data gathering. We also only obtain intermittent GPS reception due to the presence of bridges and city buildings overhead.

Due to these challenges, LOAM, LIOM, and LIO-odom all fail to produce meaningful results in this test. Similar to the problems encountered in the *Park* dataset, LIO-GPS is unable to close the loop when returning to the robot's initial position because of the drift in altitude, which further motivates our usage of loop closure factors in LIO-SAM.

F. Benchmarking Results

TABLE III: RMSE translation error w.r.t GPS

Dataset	LOAM	LIOM	LIO-odom	LIO-GPS	LIO-SAM
Park	47.31	28.96	23.96	1.09	0.96

Since full GPS coverage is only available in the *Park* dataset, we show the root mean square error (RMSE) results w.r.t to the GPS measurement history, which is treated as

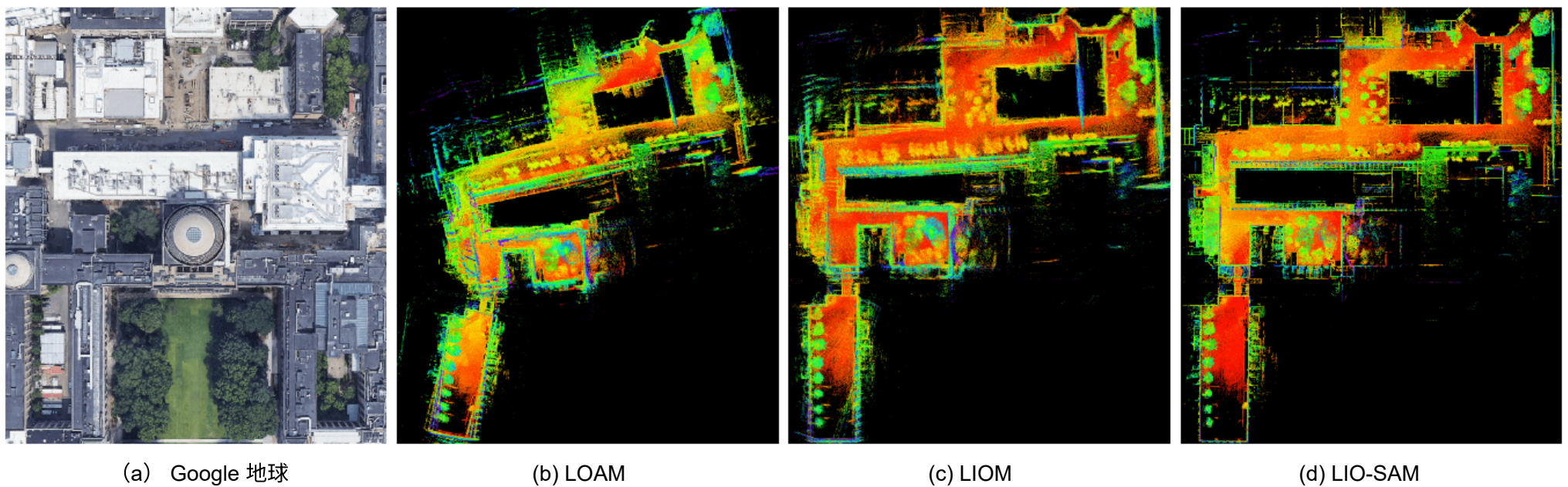


圖 4：使用 Walking 資料集之 LOAM、LIOM 和 LIO-SAM 的繪圖結果。(b) 中的 LOAM 地圖在遇到劇烈旋轉時會多次發散。LIOM 的表現優於 LOAM。然而，由於不準確的點雲配準，其地圖顯示出許多模糊的結構。LIO-SAM 生成的地圖與 Google Earth 影像一致，且未使用 GPS。

測繪區域中，GPS 接收訊號通常不可用且不準確。在濾除不一致的 GPS 測量值後，圖 5(a) 中以綠色區段顯示了 GPS 可用的區域。這些區域對應於少數未被建築物或樹木環繞的區域。

LOAM、LIO-odom、LIOGPS 和 LIO-SAM 的估計軌跡如圖 5(a) 所示。LIOM 的結果未顯示，因為它未能正確初始化並產生有意義的結果。如圖所示，與所有其他方法相比，LOAM 的軌跡顯著漂移。在沒有 GPS 資料校正的情況下，LIO-odom 的軌跡在地圖的右下角開始明顯漂移。在 GPS 資料的幫助下，LIOGPS 可以在可用時校正漂移。然而，在資料集の後半部分，GPS 資料不可用。因此，當機器人由於漂移而返回起始位置時，LIOGPS 無法閉合迴路。另一方面，LIOSAM 可以透過向圖中添加迴路閉合因子來消除漂移。LIO-SAM 的地圖與 Google Earth 影像對齊良好，如圖 5(b) 所示。當機器人返回起始位置時，所有方法的相對平移誤差如表 II 所示。

D. 公園資料集

在此測試中，我們將感測器安裝在 UGV 上，並沿著森林中的遠足小徑駕駛車輛。機器人在行駛 40 分鐘後返回其初始位置。UGV 在三種路面上行駛：柏油路、草地覆蓋的地面和泥土覆蓋的小徑。由於缺乏懸吊系統，機器人在非柏油路上行駛時會經歷低振幅但高頻率的振動。

為了模擬具挑戰性的地圖繪製情境，我們僅在機器人位於開闊區域時使用 GPS 測量，如圖 6(a) 中的綠色部分所示。這種地圖繪製情境代表了一項任務，其中機器人必須繪製多個 GPS 訊號受阻區域的地圖，並定期返回到有 GPS 可用性的區域以校正漂移。

與先前的測試結果相似，LOAM、LIOM 和 LIO-odom 存在顯著漂移，因為沒有可用的絕對校正資料。此外

LIOM 僅以 0.67 倍即時速度運行，而其他方法則以即時速度運行。儘管 LIO-GPS 和 LIO-SAM 的軌跡在水平面上重合，但它們的相對平移誤差不同（表 II）。由於沒有可靠的絕對高程測量值，LIO-GPS 在高度上存在漂移，並且在返回機器人初始位置時無法閉合迴路。LIO-SAM 沒有這個問題，因為它利用迴路閉合因子來消除漂移。

E. 阿姆斯特丹資料集

最後，我們將感測器套件安裝在船上，沿著阿姆斯特丹的運河巡航了 3 小時。儘管在此測試中感測器的移動相對平穩，但繪製運河地圖仍因多種原因而充滿挑戰。運河上的許多橋樑造成了退化場景，因為當船隻在橋下移動時，幾乎沒有有用的特徵，這類似於穿過一條長而無特徵的走廊。平面特徵的數量也顯著減少，因為地面不存在。當陽光直射感測器視野時，我們觀察到來自光達的許多錯誤偵測，這在資料收集期間約佔 20% 的時間。由於橋樑和城市建築物在頭頂上方，我們也只獲得間歇性的 GPS 接收。

由於這些挑戰，LOAM、LIOM 和 LIO-odom 在此測試中都未能產生有意義的結果。與在 Park 資料集中遇到的問題類似，LIO-GPS 無法在返回機器人初始位置時閉合迴路，因為高度漂移，這進一步促使我們在 LIO-SAM 中使用迴路閉合因子。

F. 基準測試結果

表三：相對於 GPS 的 RMSE 平移誤差

Dataset	LOAM	LIOM	LIO-odom	LIO-GPS	LIO-SAM
Park	47.31	28.96	23.96	1.09	0.96

由於完整的 GPS 覆蓋僅在 Park 資料集中可用，我們顯示了相對於 GPS 測量歷史的均方根誤差 (RMSE) 結果，該結果被視為

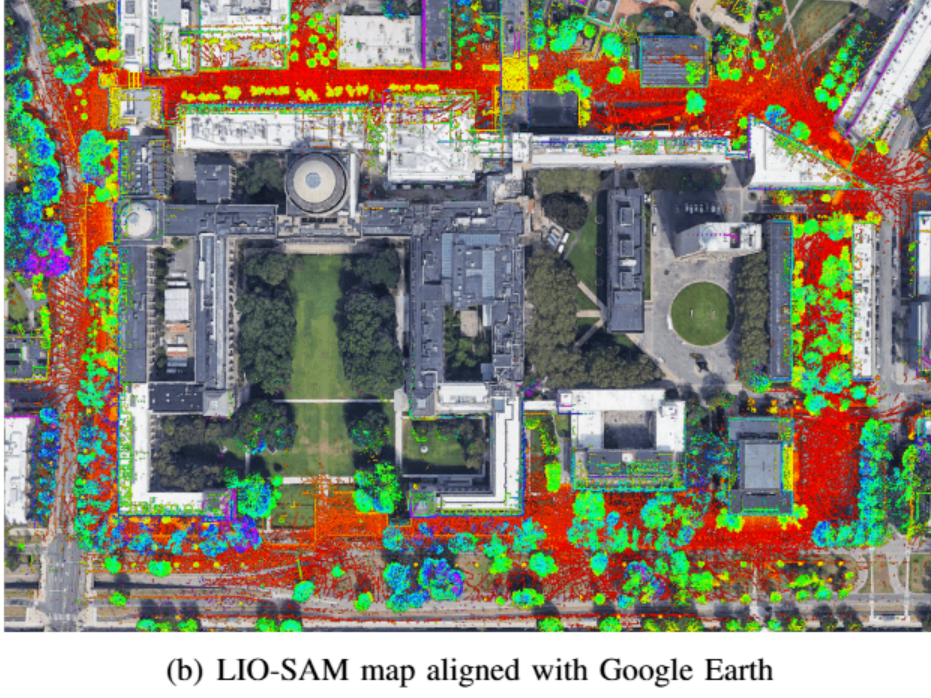
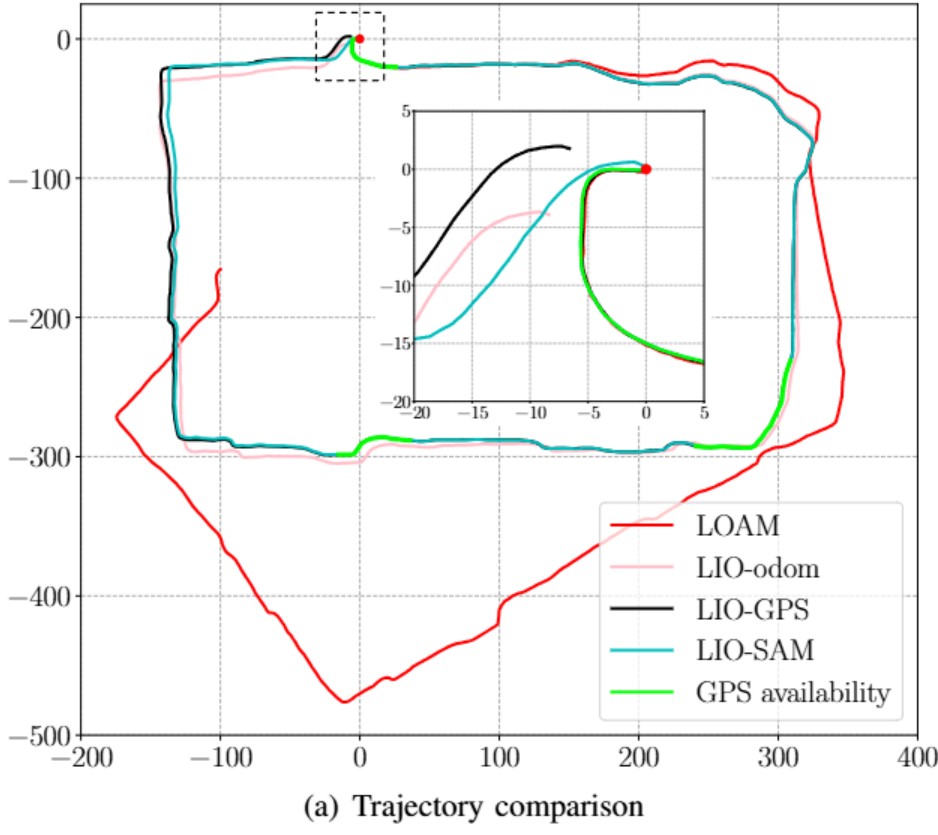


Fig. 5: Results of various methods using the *Campus* dataset that is gathered on the MIT campus. The red dot indicates the start and end location. The trajectory direction is clock-wise. LIOM is not shown because it fails to produce meaningful results.

ground truth. This RMSE error does not take the error along the z axis into account. As is shown in Table III, LIO-GPS and LIO-SAM achieve similar RMSE error with respect to the GPS ground truth. Note that we could further reduce the error of these two methods by at least an order of magnitude by giving them full access to all GPS measurements. However, full GPS access is not always available in many mapping settings. Our intention is to design a robust system that can operate in a variety of challenging environments.

The average runtime for the three competing methods to register one lidar frame across all five datasets is shown in Table IV. Throughout all tests, LOAM and LIO-SAM are forced to run in real-time. In other words, some lidar frames are dropped if the runtime takes more than 100ms when the lidar rotation rate is 10Hz. LIOM is given infinite time to process every lidar frame. As is shown, LIO-SAM uses significantly less runtime than the other two methods, which makes it more suitable to be deployed on low-power

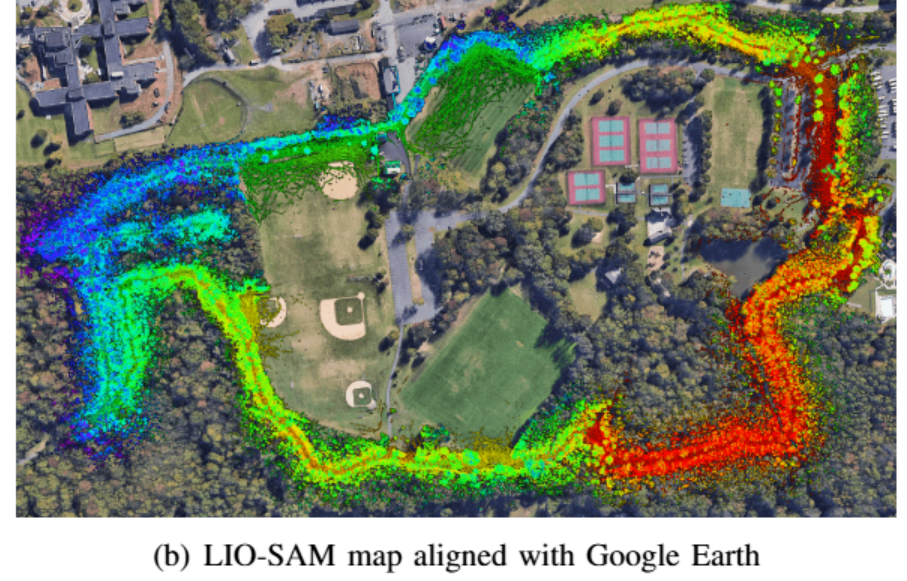
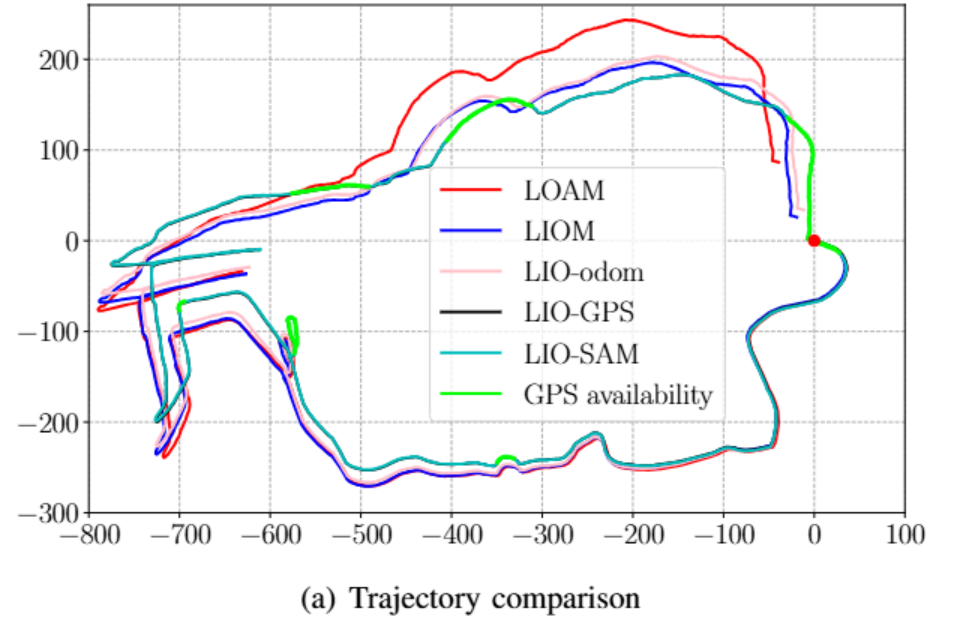


Fig. 6: Results of various methods using the *Park* dataset that is gathered in Pleasant Valley Park, New Jersey. The red dot indicates the start and end location. The trajectory direction is clock-wise.

TABLE IV: Runtime of mapping for processing one scan (ms)

Dataset	LOAM	LIOM	LIO-SAM	Stress test
Rotation	83.6	Fail	41.9	13×
Walking	253.6	339.8	58.4	13×
Campus	244.9	Fail	97.8	10×
Park	266.4	245.2	100.5	9×
Amsterdam	Fail	Fail	79.3	11×

embedded systems.

We also perform stress tests on LIO-SAM by feeding it the data faster than real-time. The maximum data playback speed is recorded and shown in the last column of Table IV when LIO-SAM achieves similar performance without failure compared with the results when the data playback speed is 1× real-time. As is shown, LIO-SAM is able to process data faster than real-time up to 13×

We note that the runtime of LIO-SAM is more significantly influenced by the density of the feature map, and less affected by the number of nodes and factors in the factor graph. For instance, the *Park* dataset is collected in a feature-rich environment where the vegetation results in a large quantity of features, whereas the *Amsterdam* dataset yields a sparser feature map. While the factor graph of the *Park* test consists of 4,573 nodes and 9,365 factors, the graph in the *Amsterdam* test has 23,304 nodes and 49,617 factors. Despite this, LIO-SAM uses less time in the *Amsterdam* test

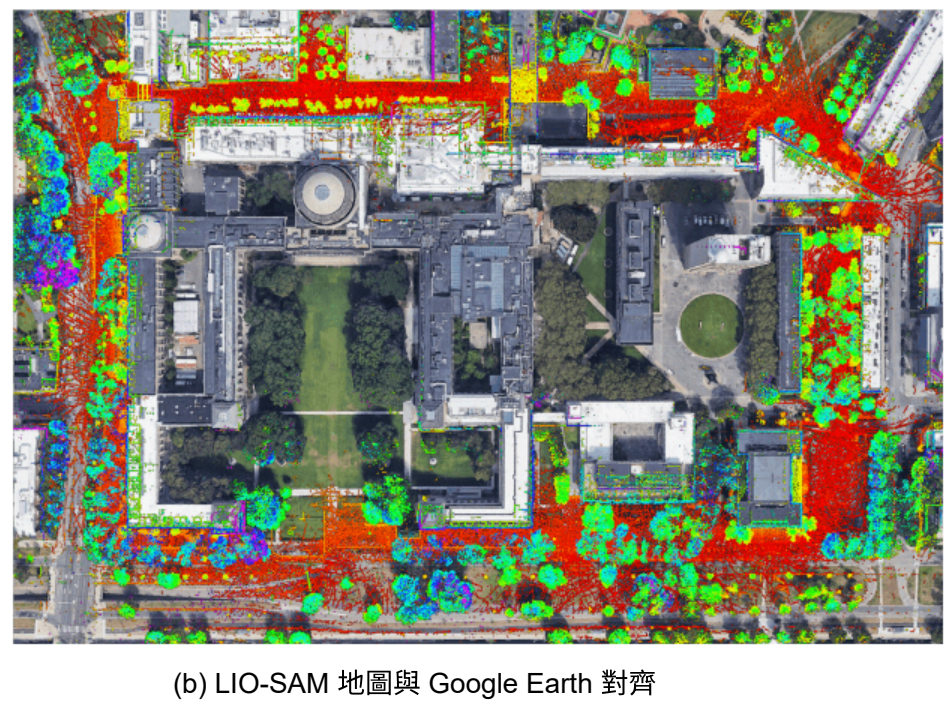
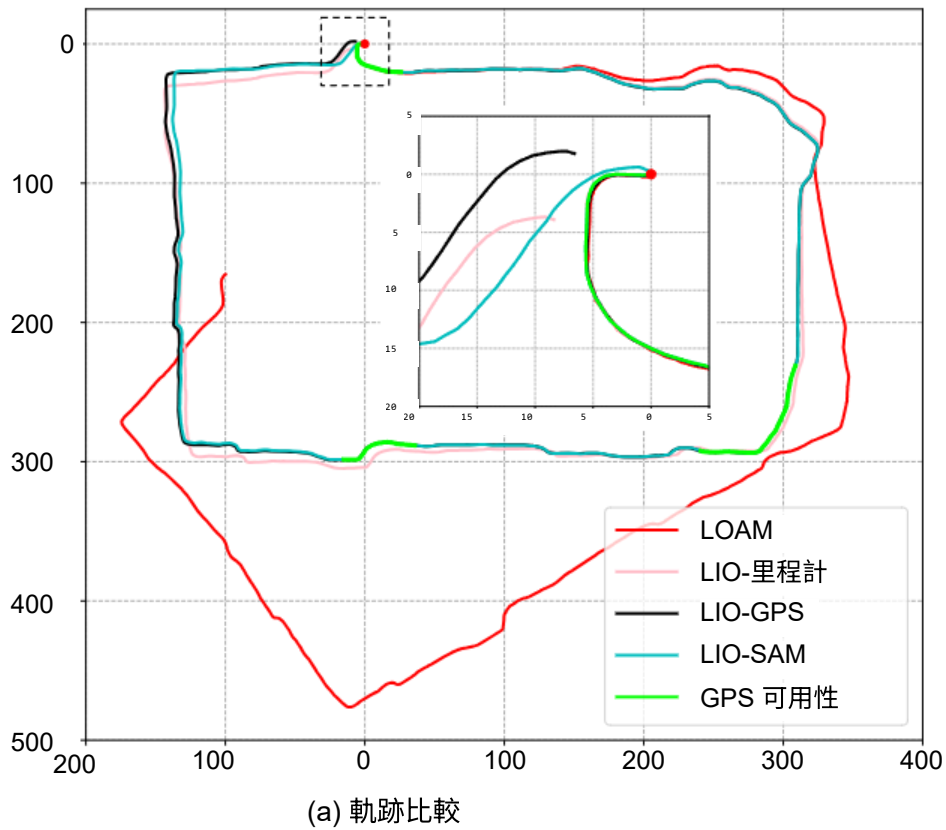


圖 5：使用在麻省理工學院校園收集的 Campus 資料集，各種方法的結果。紅點表示開始和結束位置。軌跡方向為順時針。未顯示 LIOM，因為它未能產生有意義的結果。

真實值。此 RMSE 誤差未考慮沿 z 軸的誤差。如表 III 所示，LIO-GPS 和 LIO-SAM 相對於 GPS 真實值實現了相似的 RMSE 誤差。請注意，透過讓這兩種方法完全存取所有 GPS 測量值，我們可以將其誤差至少減少一個數量級。然而，在許多地圖繪製設定中，並非總是能完全存取 GPS。我們的目的是設計一個穩健的系統，可以在各種挑戰性環境中運行。表 IV 顯示了三種競爭方法在所有五個資料集中註冊一個雷射雷達幀的平均運行時間。在所有測試中，LOAM 和 LIO-SAM 都被迫即時運行。換句話說，如果雷射雷達旋轉速率為 10Hz 時運行時間超過 100ms，則會丟棄一些雷射雷達幀。LIOM 有無限時間處理每個雷射雷達幀。如圖所示，LIO-SAM 使用的運行時間明顯少於其他兩種方法，這使其更適合部署在低功耗裝置上。

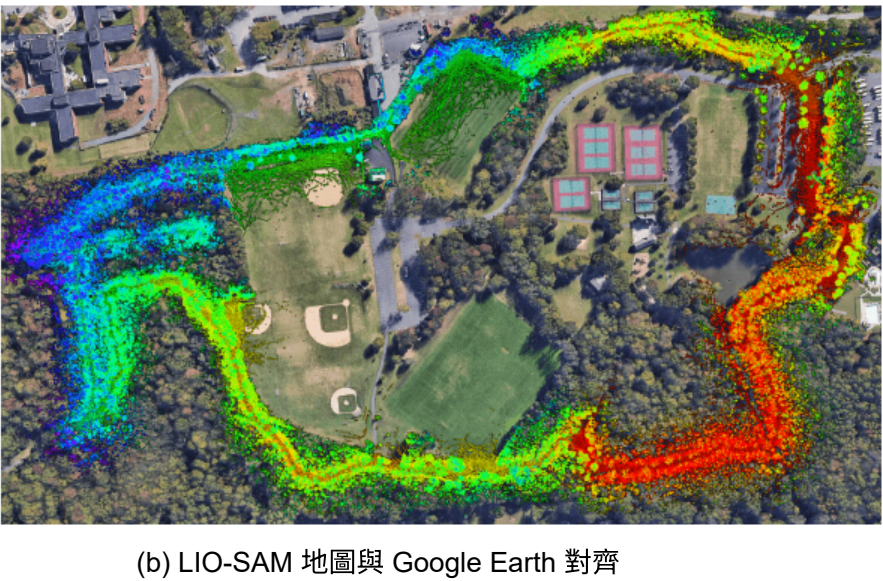
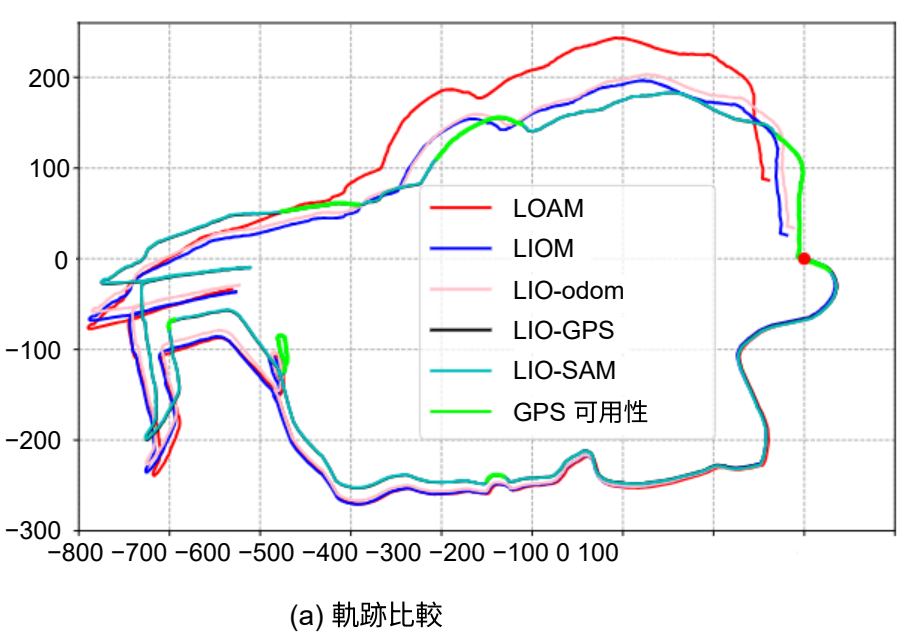


圖 6：使用在新澤西州 Pleasant Valley Park 收集的 Park 資料集，各種方法的結果。紅點表示開始和結束位置。軌跡方向為順時針。

表四：處理一次掃描的映射運行時間 (毫秒)

資料集	LOAM	LIOM	LIO-SAM	壓力測試
旋轉	83.6	失敗	41.9	13×
步行	253.6	339.8	58.4	13×
校園	244.9	失敗	97.8	10 倍
停車	266.4	245.2	100.5	9 倍
阿姆斯特丹	失敗	失敗	79.3	11 次

嵌入式系統。我們也透過以比即時更快的速度輸入資料，對 LIO-SAM 進行壓力測試。當 LIO-SAM 在不發生故障的情況下，與資料播放速度為 1 倍即時的結果相比，達到相似的效能時，最大資料播放速度會被記錄並顯示在表 IV 的最後一欄。如圖所示，LIO-SAM 能夠以比即時快達 13 倍的速度處理資料。我們注意到 LIO-SAM 的執行時間受特徵地圖密度影響更顯著，而受因子圖中節點和因子數量影響較小。例如，Park 資料集是在特徵豐富的环境中收集的，其中植被產生了大量的特徵，而 Amsterdam 資料集則產生了更稀疏的特徵地圖。儘管 Park 測試的因子圖包含 4,573 個節點和 9,365 個因子，但 Amsterdam 測試中的圖包含 23,304 個節點和 49,617 個因子。儘管如此，LIO-SAM 在 Amsterdam 測試中使用的時間更少。



Fig. 7: Map of LIO-SAM aligned with Google Earth.

as opposed to the runtime in the Park test.

V. CONCLUSIONS AND DISCUSSION

We have proposed LIO-SAM, a framework for tightly-coupled lidar inertial odometry via smoothing and mapping, for performing real-time state estimation and mapping in complex environments. By formulating lidar-inertial odometry atop a factor graph, LIO-SAM is especially suitable for multi-sensor fusion. Additional sensor measurements can easily be incorporated into the framework as new factors. Sensors that provide absolute measurements, such as a GPS, compass, or altimeter, can be used to eliminate the drift of lidar inertial odometry that accumulates over long durations, or in feature-poor environments. Place recognition can also be easily incorporated into the system. To improve the real-time performance of the system, we propose a sliding window approach that marginalizes old lidar frames for scan-matching. Keyframes are selectively added to the factor graph, and new keyframes are registered only to a fixed-size set of sub-keyframes when both lidar odometry and loop closure factors are generated. This scan-matching at a local scale rather than a global scale facilitates the real-time performance of the LIO-SAM framework. The proposed method is thoroughly evaluated on datasets gathered on three platforms across a variety of environments. The results show that LIO-SAM can achieve similar or better accuracy when compared with LOAM and LIOM. Future work involves testing the proposed system on unmanned aerial vehicles.

REFERENCES

- [1] J. Zhang and S. Singh, "Low-drift and Real-time Lidar Odometry and Mapping," *Autonomous Robots*, vol. 41(2): 401-416, 2017.
- [2] A. Geiger, P. Lenz, and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite", *IEEE International Conference on Computer Vision and Pattern Recognition*, pp. 3354-3361, 2012.
- [3] P.J. Besl and N.D. McKay, "A Method for Registration of 3D Shapes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14(2): 239-256, 1992.
- [4] A. Segal, D. Haehnel, and S. Thrun, "Generalized-ICP," *Proceedings of Robotics: Science and Systems*, 2009.
- [5] W.S. Grant, R.C. Voorhies, and L. Itti, "Finding Planes in LiDAR Point Clouds for Real-time Registration," *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 4347-4354, 2013.
- [6] M. Velas, M. Spanel, and A. Herout, "Collar Line Segments for Fast Odometry Estimation from Velodyne Point Clouds," *IEEE International Conference on Robotics and Automation*, pp. 4486-4495, 2016.
- [7] T. Shan and B. Englot, "LeGO-LOAM: Lightweight and Ground-optimized Lidar Odometry and Mapping on Variable Terrain," *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 4758-4765, 2018.
- [8] T. Shan, J. Wang, K. Doherty, and B. Englot, "Bayesian Generalized Kernel Inference for Terrain Traversability Mapping," *In Conference on Robot Learning*, pp. 829-838, 2018.
- [9] S. Lynen, M.W. Achtelik, S. Weiss, M. Chli, and R. Siegwart, "A Robust and Modular Multi-sensor Fusion Approach Applied to MAV Navigation," *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 3923-3929, 2013.
- [10] S. Yang, X. Zhu, X. Nian, L. Feng, X. Qu, and T. Mal, "A Robust Pose Graph Approach for City Scale LiDAR Mapping," *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1175-1182, 2018.
- [11] M. Demir and K. Fujimura, "Robust Localization with Low-Mounted Multiple LiDARs in Urban Environments," *IEEE Intelligent Transportation Systems Conference*, pp. 3288-3293, 2019.
- [12] Y. Gao, S. Liu, M. Atia, and A. Noureldin, "INS/GPS/LiDAR Integrated Navigation System for Urban and Indoor Environments using Hybrid Scan Matching Algorithm," *Sensors*, vol. 15(9): 23286-23302, 2015.
- [13] S. Hening, C.A. Ippolito, K.S. Krishnakumar, V. Stepanyan, and M. Teodorescu, "3D LiDAR SLAM integration with GPS/INS for UAVs in urban GPS-degraded environments," *AIAA Infotech@Aerospace Conference*, pp. 448-457, 2017.
- [14] C. Chen, H. Zhu, M. Li, and S. You, "A Review of Visual-Inertial Simultaneous Localization and Mapping from Filtering-Based and Optimization-Based Perspectives," *Robotics*, vol. 7(3):45, 2018.
- [15] C. Le Gentil, T. Vidal-Calleja, and S. Huang, "IN2LAMA: Inertial Lidar Localisation and Mapping," *IEEE International Conference on Robotics and Automation*, pp. 6388-6394, 2019.
- [16] C. Qin, H. Ye, C.E. Pranata, J. Han, S. Zhang, and Ming Liu, "R-LINS: A Robocentric Lidar-Inertial State Estimator for Robust and Efficient Navigation," *arXiv:1907.02233*, 2019.
- [17] H. Ye, Y. Chen, and M. Liu, "Tightly Coupled 3D Lidar Inertial Odometry and Mapping," *IEEE International Conference on Robotics and Automation*, pp. 3144-3150, 2019.
- [18] F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," *Foundations and Trends in Robotics*, vol. 6(1-2): 1-139, 2017.
- [19] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J.J. Leonard, and F. Dellaert, "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," *The International Journal of Robotics Research*, vol. 31(2): 216-235, 2012.
- [20] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," *IEEE Transactions on Robotics*, vol. 33(1): 1-21, 2016.
- [21] T. Moore and D. Stouch, "A Generalized Extended Kalman Filter Implementation for The Robot Operating System," *Intelligent Autonomous Systems*, vol. 13: 335-348, 2016.
- [22] G. Kim and A. Kim, "Scan Context: Egocentric Spatial Descriptor for Place Recognition within 3D Point Cloud Map," *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 4802-4809, 2018.
- [23] J. Guo, P. VK Borges, C. Park, and A. Gawel, "Local Descriptor for Robust Place Recognition using Lidar Intensity," *IEEE Robotics and Automation Letters*, vol. 4(2): 1470-1477, 2019.
- [24] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A.Y. Ng, "ROS: An Open-source Robot Operating System," *IEEE ICRA Workshop on Open Source Software*, 2009.
- [25] T. Qin, P. Li, and S. Shen, "Vins-mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," *IEEE Transactions on Robotics*, vol. 34(4): 1004-1020, 2018.



圖 7：LIO-SAM 地圖與 Google Earth 對齊。

Park 資料集是在一個功能豐富的环境中收集的，其中植被產生了大量的特徵。這與 Park 測試中的運行時間相反。

V. CD

我們提出了 LIO-SAM，這是一個透過平滑和映射實現緊密耦合雷射雷達慣性里程計的框架，用於在複雜環境中執行即時狀態估計和映射。透過在因子圖上建立雷射雷達慣性里程計，LIO-SAM 特別適合多感測器融合。額外的感測器測量可以輕鬆地作為新因子整合到框架中。提供絕對測量的感測器，例如 GPS、指南針或高度計，可用於消除長時間累積或在特徵貧乏環境中雷射雷達慣性里程計的漂移。地點識別也可以輕鬆整合到系統中。為了提高系統的即時性能，我們提出了一種滑動視窗方法，該方法將舊的雷射雷達幀邊緣化以進行掃描匹配。關鍵幀被選擇性地添加到因子圖中，並且只有當雷射雷達里程計和閉環因子都生成時，新的關鍵幀才會註冊到固定大小的子關鍵幀集。這種在局部而非全局範圍內的掃描匹配有助於 LIO-SAM 框架的即時性能。所提出的方法在三個平台上收集的各種環境數據集上進行了徹底評估。結果表明，與 LOAM 和 LIOM 相比，LIO-SAM 可以實現相似或更好的精度。未來的工作包括在無人機上測試所提出的系統。

R

[1] J. Zhang and S. Singh, "Low-drift and Real-time Lidar Odometry and Mapping," Autonomous Robots, vol. 41(2): 401-416, 2017. [2] A. Geiger, P. Lenz, and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite", IEEE International
電腦視覺與圖樣辨識會議，頁 3354-3361，2012。

- [3] P.J. Besl 和 N.D. McKay, 「一種 3D 形狀配準的<方法>」, IEEE 圖樣分析與機器智慧學報, 卷。 14(2): 239-256, 1992.
- [4] A. Segal、D. Haehnel 和 S. Thrun, 「廣義 ICP」, 機器人學: 科學與系統會議錄, 2009 年。[5] W.S. Grant、R.C. Voorhies 和 L. Itti, 「在 LiDAR 點雲中尋找平面以進行即時配準」, IEEE/RSJ 智慧型機器人與系統國際會議, 頁 4347-4354, 2013 年。[6] M. Velas、M. Spanel 和 A. Herout, 「用於從 Velodyne 點雲快速估計里程計的領線段」, IEEE 機器人與自動化國際會議, 頁 4486-4495, 2016 年。[7] T. Shan 和 B. Englot, 「LeGO-LOAM: 在可變地形上的輕量級和地面優化雷射雷達里程計和映射」, IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 4758-4765, 2018.
- [8] T. Shan、Q. Wang, K. Doherty, and B. Englot, "Bayesian Generalized Kernel Inference for Terrain Traversability Mapping," In Conference on Robot Learning, pp. 829-838, 2018. [9] S. Lynen, M.W. Achtelik, S. Weiss, M. Chli, and R. Siegwart, "A Robust and Modular Multi-sensor Fusion Approach Applied to MAV Navigation," IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 3923-3929, 2013.
- [10] S. Yang、X. Zhu、X. Nian, L. Feng, X. Qu, and T. Mal, "A Robust Pose Graph Approach for City Scale LiDAR Mapping," IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 1175-1182, 2018.
- [11] M. Demir 和 K. Fujimura, 「Robust Localization with Low-Mounted Multiple LiDARs in Urban Environments」, IEEE Intelligent Transportation Systems Conference, pp. 3288-3293, 2019.
- [12] C. S. Goh, S. M. Ali, and A. Noureldin, "An GPS/LiDAR Integrated Navigation System for Urban and Indoor Environments using Hybrid Scan Matching Algorithm," Sensors, vol. 15(9): 23286-23302, 2015.
- [13] S. Hening, C.A. Ippolito, K.S. Krishnakumar, V. Stepanyan, and M. Teodorescu, "3D LiDAR SLAM integration with GPS/INS for UAVs in urban GPS-degraded environments," AIAA Infotech@Aerospace Conference, pp. 448-457, 2017.
- [14] C. Chen, H. Zhu, M. Li, and S. You, "A Review of Visual-Inertial Simultaneous Localization and Mapping from Filtering-Based and Optimization-Based Perspectives," Robotics, vol. 7(3):45, 2018.
- [15] C. Le Gentil, T. Vidal-Calleja, and S. Huang, "IN2LAMA: Inertial Lidar Localisation and Mapping," IEEE International Conference on Robotics and Automation, pp. 6388-6394, 2019.
- [16] C. Qin, H. Ye, C.E. Pranata, J. Han, S. Zhang, and Ming Liu, "R-LINS: 一種以機器人為中心的雷射雷達慣性狀態估計器，用於穩健高效的導航," arXiv:1907.02233, 2019.
- [17] H. Ye、Y. Chen 和 M. Liu, 「緊密耦合的 3D 雷射雷達慣性里程計與建圖」, IEEE 國際機器人與自動化會議, 頁 3144-3150, 2019。
- [18] F. Dellaert 和 M. Kaess, 「用於機器人感知的因子圖」, Foundations and Trends in Robotics, vol. 6(1-2): 1-139, 2017.
- [19] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J.J. Leonard, and F. Dellaert, "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," The International Journal of Robotics Research, vol. 31(2): 216-235, 2012.
- [20] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE Transactions on Robotics, 第 33(1) 卷: 1-21, 2016 年。
- [21] T. Moore 和 D. Stouch, 「A Generalized Extended Kalman Filter Implementation for The Robot Operating System」, Intelligent Autonomous Systems, 第 13 卷: 335-348, 2016 年。
- [22] G. Kim 和 A. Kim, 「Scan Context: Egocentric Spatial Descriptor for Place Recognition within 3D Point Cloud Map」, IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 4802-4809, 2018.
- [23] J. Guo, P. VK Borges, C. Park, and A. Gawel, "Local Descriptor for Robust Place Recognition using Lidar Intensity," IEEE Robotics and Automation Letters, vol. 4(2): 1470-1477, 2019.
- [24] M. Quigley、K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler 和 A.Y. Ng, 「ROS: 一個開源機器人作業系統」, IEEE ICRA 開源軟體研討會, 2009 年。
- [25] T. Qin、P. Li 和 S. Shen, 「Vins-mono: 一個穩健且多功能的單眼視覺慣性狀態估計器」, IEEE Transactions on Robotics, 第 34(4) 卷: 1004-1020, 2018 年。